Part I - Periodic Framework Topology, Rings, Natural Tilings, and Structural Semantics

Species-Independent Structural Architecture for mdstats

mdstats

2026-07-30 (revision 39 - optional MLFF profile boundary; structural algorithms unchanged)

Contents

Purpose and status 4
Theoretical background 8
From an atomic configuration to a graph 8
Periodic nets and finite quotient graphs 9
Framework projection and decorated edges 9
Cycles, local rings, and primitive rings 9
Topology, embedding, tiling, and chemical interpretation 10
Design goals 11
Core separation of concerns 12
Frame semantics and topology semantics 12
Geometric neighborhood and atomic connectivity 12
Connectivity scope and framework participation 13
Persistent identity scope and dynamic region membership 14
Relationship to RDF, coordination, and angle statistics 15
Topology consistency categories 15
Topology classes and trajectory segments 15
General framework model 16
Atomic connectivity source 16
Atomic graph and projected framework graph 16
Undirected adjacency and orientation-aware path decoration 17
Decorated periodic multigraph 17
Unrestricted framework degree 17
Bounded linker discovery 18
Mapping-dependent ring size 18
Atomic connectivity architecture 18
Connectivity scope 18
Definition objects and evaluated results 19
Distance connectivity 19
Hysteretic trajectory connectivity 19
Reference connectivity for ensembles 19
Ensemble-consensus connectivity 19
Connectivity state compression 19
Scientific naming 20
Module architecture 20
`atomic_connectivity.py` 21
Responsibility 21
Primary inputs 21
Primary outputs 22
Owned operations 22
Explicit non-responsibilities 22
`framework_topology.py` 22

Responsibility 22
Primary inputs 22
Primary outputs 23
Owned operations 23
Explicit non-responsibilities 23
Important invariants 23
topology_catalog.py 24
Responsibility 24
Primary API 24
Exact class identity 24
Trajectories and ensembles 24
Compression and provenance 25
Explicit non-responsibilities 25
region_membership.py 25
Responsibility 25
Primary inputs 25
Primary outputs 25
Analysis levels 25
Fixed geometric regions 26
Reference-aligned regions 26
Topology-aware dynamic regions 26
Important invariants 26
Explicit non-responsibilities 26
primitive_ring.py 26
Responsibility 26
Primitive definition 26
Periodic covering graph 27
Bounded periodic completeness theorem 27
Translation-orbit representative 27
Even cycles 27
Odd cycles 27
Finite reduction 27
Conclusion 28
Candidate generation and classification 28
Resource and provenance semantics 28
Algorithmic attribution 28
Implementation status 29
Explicit non-responsibilities 29
_periodic_graph.py 29
periodic_net_view.py 29
periodic_barycentric.py 30
periodic_net_embedding.py 30
periodic_cycle.py 31
primitive_ring_index.py 33
primitive_ring_cancellation.py 34
net_symmetry.py 35
primitive_ring_symmetry.py 35
ring_strength.py 35
_periodic_spatial.py 36
Reuse of the existing Verlet validity theorem 38
periodic_edge_intersection.py 39
face_candidates.py 39

Linking and catenation semantics ... 39
periodic_cell_complex.py ... 40
natural_tiling.py ... 41
natural_tiling_search.py ... 42
ring_geometry.py ... 43
Responsibility ... 43
ring_site.py ... 43
Responsibility ... 43
tiling_geometry.py ... 43
Responsibility ... 43
cage.py ... 43
Responsibility ... 43
Identity, fingerprints, and reproducibility ... 44
Exact labeled equality ... 44
Canonical schema and stable digests ... 44
Immutability ... 44
Periodic gauge normalization ... 44
Atom identity contract ... 44
Topology fingerprint ... 44
Ring canonical key ... 45
Global ring catalog across topologies ... 45
Cross-cutting provenance ... 45
Standard analysis workflows ... 46
Nonreactive trajectory ... 46
Fixed-topology ensemble ... 46
Partitioned reactive trajectory ... 46
Multi-topology ensemble ... 47
Per-frame topology analysis ... 47
Heterogeneous solid-liquid or solid-solid interface ... 47
Validation strategy ... 48
Implemented-layer regression ... 48
Net-view tests ... 48
Stage-5 prototype tests ... 48
Symmetry tests ... 48
Strength tests ... 48
Stage-7R boundary tests ... 49
Periodic-net-embedding tests ... 49
Periodic spatial broad-phase and cache tests ... 49
Embedded-face tests ... 49
Periodic-cell-complex tests ... 50
Natural-tiling tests ... 50
LTA domain validation ... 50
Implementation sequence ... 50
Stages 1-4R - implemented ... 50
Stage 5-P0/P1 - exact ring-placement infrastructure - implemented ... 50
Stage 5-P2 - exact automorphism-induced ring occurrence action - implemented ... 50
Stage 5-P3 - exact finite GF(2) primitive-ring support cancellation - implemented ... 51
Stage 5-R - API hygiene and shared periodic infrastructure - implemented ... 51
Stage 5.1 - explicit net-view contract (**implemented**) ... 51
Stage 5.2 - private consumer prototypes ... 52
Stage 5.3 - stable identity and lightweight views ... 52
Stage 5.4 - extract shared private helpers and freeze API ... 52

Stage 6A - view-bound explicit automorphism validation (**implemented**) 52
Stage 6B - explicit-generator periodic symmetry group - implemented and hardened 52
Stage 6C - automatic periodic-net symmetry discovery - implemented 52
Stage 7 - bounded strong-ring classification - implemented 53
Stage 7R - certification and persistence consolidation - implemented 53
Stage 8A - authoritative periodic-net embedding - implemented 53
Stage 8B - periodic extended-object broad phase - implemented 53
Stage 8C - embedded face placements - implemented 53
Stage 9 - periodic cell complex and partition certificate - implemented 53
Stage 10 - natural-tiling orchestration 54
Stage 10A - candidate and properness certification - implemented 54
Stage 10B - natural face selection and local splitting - implemented 54
Stage 10C - ring-bound refinement - implemented 54
Stage 10D - LTA end-to-end gate - implemented 54
Stage 11 structural completion and Part II handoff 55
Part I implemented structural stages 55
Normative handoff contract 55
Part II ownership 55
Deferred features 55
Accepted architectural decisions 56
Theoretical and algorithmic references 59
Context-restoration checklist 61
Final architecture summary 62
Optional MLFF material-profile boundary 63
MLFF optional-extension integration 63

Purpose and status

This manual is **Part I** of the Stage 11 architecture. It owns the immutable, species-independent structural coordinate system: periodic connectivity, primitive rings, symmetry, embeddings, natural tilings, tile/cage/window geometry, persistent and compatible-frame ring geometry, atom-resolved serrated ring boundaries, and framework semantics through Stage 11D.

Revision 38 removes the duplicated statistical-site and kinetic roadmap that had accumulated in this manual. Registered trajectory analysis, species-density inference, site validation, temporal segmentation, observed paths/networks, and all deferred kinetic models are owned exclusively by `stage11_site_kinetics_architecture.{md,pdf}` (**Part II**). The two manuals share a single boundary: Part I exports stable structural identities and registered structural descriptors; Part II consumes them without redefining the framework.

This revision is documentation and private-helper consolidation only. It does not change any structural scientific identity or Stage 11E8a conclusion.

Revision 37 established the data-driven Stage 11 site-state architecture and moved its complete physical plan into the dedicated Part II manual. Revision 38 finishes that separation by removing the stale duplicate roadmap from Part I.

Revision 36 implements Stage 11B compatible-frame natural-tile geometry. `tiling_geometry_frames.py` maps the fixed scientific faces, windows, tile shells, labels, and translation-labelled adjacency of one certified natural tiling onto selected trajectory or ensemble frames. It does not rediscover or mutate ring, face, or tile identity. Every frame is independently projected through the supplied atomic-connectivity and framework-mapping policy; only an exact `FrameworkTopology.graph_digest` match is eligible for geometry.

The module replays the complete periodic integer gauge used by the existing connectivity and framework-projection pipeline: physical atomic minimum-image shifts, atomic-state canonicalization, relevant-subgraph normalization, and final projected-framework normalization. A trajectory receives one global integer placement that aligns a deterministic anchor with its unwrapped coordinate; an ensemble is wrapped independently. Fixed source ring walks and attaching translations then reconstruct every lifted tile side in the instantaneous cell. Thermally nonplanar faces are retained as deterministic boundary-

center fan surfaces with explicit planarity diagnostics; they never redefine scientific faces. Tile volumes and first moments are accumulated from oriented fan tetrahedra and must close to the instantaneous cell volume.

Per-frame results distinguish `MAPPED`, `TOPOLOGY_MISMATCH`, `CONNECTIVITY_GEOMETRY_MISMATCH`, and `DEGENERATE_GEOMETRY`. Unresolved frames contain no partial geometry, and frame-aligned metric series use `NaN` for them. The persistent catalog independently binds the reference tiling, selected collection geometry, connectivity states, and topology catalog and is loaded by deterministic source replay. Chemistry-aware radii, explicit linker surfaces, maximum-clearance searches, free-volume connectivity, and dynamic crossing events remain deferred.

Revision 35 implements the first Stage-11 tile-geometry and cage-accessibility backend. `tiling_geometry.py` realizes every scientific tile shell in the exact source-bound unit-volume embedding, verifies strict planar face convexity and convex supporting halfspaces, and computes exact rational tile volume and volume centroid. Derived Cartesian descriptors include face areas, perimeters, deterministic aperture witnesses, tile surface area, diameter, equivalent-sphere radius, and Wadell sphericity. Every scientific face orbit becomes one `TopologicalWindow` with its two translated tile-side incidences and two reverse directed adjacency arcs; self-image tile adjacency remains explicit rather than being collapsed.

`cage.py` separates `NaturalTile`, `TopologicalWindow`, accessible cage witnesses, and accessible portal witnesses. Periodic spherical obstacles are explicit inputs. A witness is certified only when its complete metric-derived nearest-image search clears the probe radius; a blocked witness remains `WITNESS_BLOCKED_UNRESOLVED` and is never promoted to global inaccessibility. The accessible quotient graph retains lattice voltages and reports rank zero through three as isolated cages, one-dimensional channels, two-dimensional layers, or three-dimensional networks. `build_lta_natural_tiling_reference()` exposes the already certified Stage-10D LTA sources without creating a second persistent tiling identity. The reference backend remains convex-tile, spherical-probe, and witness-sufficient only. Stage 11B now maps its fixed scientific identities onto compatible arbitrary frames; full free-volume search remains deferred.

Revision 34 implements Stage 10D, the exact LTA natural-tiling ground gate. `lta_natural_tiling.py` accepts the unlabeled 48-vertex, 96-edge, degree-four LTA quotient, discovers its complete order-96 periodic automorphism group, builds the authoritative rational embedding, and independently rebuilds primitive rings, depth-one bounded strength, and exact ring-polygon geometry at $K = 8, 10, 12$. Scientific faces are exactly the bounded-strong rings whose authoritative polygons are strictly convex and planar. This retains 36 four-rings, 16 six-rings, and 6 eight-rings at every tested bound; the 32 new strong twelve-rings at $K = 12$ are recorded explicitly as nonplanar exclusions.

Exact cyclic face-ray order around each lifted framework edge propagates translation-labelled face sides into ten finite genus-zero tile shells. The resulting quotient complex has $(48, 96, 58, 10)$ cells and recovers six $[4^6]$, two $[4^{6.6^8}]$, and two $[4^{12}.6^{8.8^6}]$ tiles, hence ratio $3 : 1 : 1$. Properness is proved on a certified generating set whose multiplication- table closure recovers all 96 automorphisms. A separate exact convex periodic partition certificate verifies supporting halfspaces, strict interior witnesses, positive rational volumes summing to one, and pairwise periodic interior disjointness by face-normal and edge-cross-edge separating axes. At higher bounds, downstream evidence is reused only after independent ring/strength/geometry rebuilds prove exact selected-ring stable-key equality. Generic automatic master-refinement construction remains deferred; Stage 11 now begins the separate tile-geometry, cage, portal, and accessibility branch.

Revision 33 implements Stage 10C primitive-ring-bound refinement. `natural_tiling_refinement.py` treats the requested primitive-ring upper bound as a transactional invalidation boundary: every source-bound ring index, induced ring action, strength catalog, face family, compatibility system, master complex, partition certificate, Stage-10B search, and Stage-10A catalog must be rebuilt against the current primitive-ring catalog. Reuse is rejected when any downstream source digest names an earlier bound.

Each complete rebuild is reduced to canonical stable scientific records. Ring identity uses `PrimitiveRingKey`; face identity uses the embedding, ring key, translated placement, and orientation; complex identity rewrites face and tile incidence through stable keys and discards dense local IDs. Consecutive bounds are compared as added, removed, or modified records. Disappearance of a primitive ring under two complete increasing bounds is an invalid monotonicity violation. Unresolved upper bounds cannot be reported as stable. The report also identifies only a stable **tested suffix** of the supplied bound sequence and makes no claim about untested larger rings. Stage 10D remains the LTA end-to-end gate; automatic master-refinement construction remains deferred.

Revision 32 implements Stage 10B natural face selection and local splitting over one exact master refinement. `natural_tiling_search.py` computes full-symmetry orbits of the master scientific faces, admits only boundedly strong

orbits, exhausts every nonempty symmetry-closed subset within declared resources, and prunes fixed-witness hard or unresolved constraints before reconstruction. Omitted master interfaces become translation-labelled tetrahedral adjacencies. A quotient component is accepted as a finite lifted tile only when every closed walk has zero accumulated lattice translation; nonzero voltage cycles certify periodic slabs, channels, or other noncompact components.

For each surviving selection, the module reassigns the unchanged master tetrahedra to translated final tile placements, reconstructs complete oriented scientific face sides, rebuilds the Stage-9 chain complex, re-runs exact periodic partition certification, and applies Stage-10A properness and eligibility. Only inclusion-maximal valid strong-ring splittings survive; incomparable crossing alternatives remain explicit. The backend is complete only relative to the supplied master tetrahedral arrangement and its fixed witness assignment. Automatic master-refinement construction, primitive-bound rebuild, and the LTA end-to-end gate remain Stages 10C–10D.

Revision 31 implements Stage 10A natural-tiling candidate and properness certification. `natural_tiling.py` maps every representative of one complete Stage-7R periodic-net automorphism group onto mesh-independent scientific face orbits and translation-labelled tile attaching maps. The action retains exact lattice shifts and orientation signs and is checked against the full normalized group multiplication table, including the common translation removed from each representative product. Auxiliary face triangulations and tetrahedral partition meshes remain evidence and are excluded from scientific tiling identity.

Candidate eligibility is now multidimensional: primitive-ring completeness, complete symmetry, bounded local strength, embedded-face evidence, witness compatibility, Stage-9 complex validity, exact partition certification, and properness remain independent states. Missing evidence stays `UNRESOLVED`; certified weak faces or a failed full-symmetry image make a candidate ineligible. `NaturalTilingCatalog` deduplicates equal scientific complexes across auxiliary evidence, preserves unresolved and rejected candidates, reports `NONE/UNIQUE/MULTIPLE` over eligible identities only, and defines essential rings only from accepted face sets. Natural face selection, local splitting, ring-bound refinement, and the LTA end-to-end gate remain Stages 10B–10D.

Revision 30 implements Stage 9 translation-labelled periodic cell complexes and exact periodic partition certification. `PeriodicCellComplex` now owns the finite scientific $0/1/2/3$-cell quotient, integer coefficients, explicit lattice shifts, and the formal operators ∂_1, ∂_2, and ∂_3. Construction verifies both chain identities, two translated tile-side incidences per face orbit, three-torus Euler characteristic zero, and connected orientable nonbranching genus-zero lifted boundaries for every proposed tile orbit. Local face-sector propagation remains deliberately unimplemented: Stage 9 validates caller-supplied shells rather than presenting an unproved discovery heuristic as a theorem.

The separate `PeriodicPartitionCertificate` consumes an explicit periodic tetrahedral decomposition. It uses the complete Stage-8B image-labelled broad phase and exact rational separating-axis predicates to allow shared boundaries while rejecting improper, containment, and coincident interior overlap. Every periodic auxiliary facet must pair exactly twice with opposite orientation; every scientific interface must conform to one selected Stage-8C witness triangle; the induced scientific tile shell must reproduce ∂_3 exactly; and positive exact tile volumes must sum to one only after nonoverlap and closed-facet checks. Scientific complex identity remains independent of arbitrary auxiliary mesh details. Both principal records are deterministically source-replay verified.

Revision 29 implements Stage 8C embedded face placements. Scientific `FacePlacement` identity is now independent of auxiliary `FaceEmbeddingWitness` triangulations. The first exact backend exhausts every boundary-vertex triangulation within declared Catalan/resource bounds, rejects degenerate and periodically self-intersecting PL disks, records framework penetration separately from disk embeddedness, and preserves complete source identity to the Stage-8A embedding, Stage-8B edge certificate, and primitive-ring catalog.

Exact rational segment–triangle and triangle–triangle predicates now support periodic framework penetration, algebraic ring–surface intersection, and particular-witness surface compatibility. Nonzero oriented intersection certifies linking; intersection of particular disks proves only witness incompatibility; disjoint embedded disks provide a bounded unlinking witness; and all degenerate or finite-search failures remain explicit `UNRESOLVED` states. A finite constraint system records unary, pairwise, caller-supplied higher-order, scientific symmetry, and witness-equivariance relations. Face, pair, and compatibility certificates are deterministically source-replay verified during deserialization.

Revision 28 implements the first periodic extended-object broad phase and exact global straight-edge intersection certificate. `_periodic_spatial.py` accepts continuous lifted fractional AABB supports, derives a complete finite translation stencil from their actual bounds, and returns canonical image-labelled candidates using either exhaustive direct enumeration or a

deterministic linked-cell subdivision. Candidate identity retains `(object_i, object_j, image_shift)`, including nonzero self images.

`periodic_edge_intersection.py` consumes the authoritative Stage-8A embedding and tests every surviving periodic segment pair exactly over rational fractional coordinates. Invertibility of the lattice map makes segment incidence affine invariant, so no floating Cartesian tolerance enters the scientific decision. A contact is allowed only when both segment endpoints denote the same exact `LiftedVertexRef`. Proper crossings, endpoint-on-interior contacts, contacts between distinct lifted vertices, and positive-length collinear overlaps invalidate the straight-edge realization. The source-bound certificate is independently replayed during deserialization. Na-LTA is globally certified intersection-free under its 96-operation authoritative embedding.

Revision 27 implements the first authoritative Euclidean realization of one exact `PeriodicNetView`. `PeriodicNetEmbedding` combines the collision-free rational `PeriodicBarycentricPlacement` with the complete discovered `PeriodicNetSymmetry` and an exact positive-definite lattice metric. The first projected-edge model is an explicit straight segment between authoritative net vertices; expanded atomic linker paths remain chemical provenance rather than silent face-boundary geometry.

The lattice metric is derived from the exact second moment of all projected quotient-edge vectors. For

$$\mathbf{d}_e = \mathbf{x}_j + \boldsymbol{\delta}_e - \mathbf{x}_i, \qquad C = \sum_e \mathbf{d}_e \mathbf{d}_e^T,$$

the stored metric shape is the primitive integral normalization of

$$G = C^{-1}.$$

Because complete automorphisms permute the edge vectors as $\mathbf{d}_{g(e)} = \pm A_g \mathbf{d}_e$, the implementation verifies exactly that

$$A_g^T G A_g = G.$$

The construction is also covariant under unimodular lattice-basis changes, unlike a metric obtained by averaging an arbitrary identity form in the current source basis. Numerical Cartesian coordinates use the deterministic lower-triangular Cholesky factor of the unit-volume normalization of G. The exact scientific record remains the rational fractional placement plus integral Gram matrix.

Stage 8A certifies distinct periodic vertices, positive edge lengths, absence of coincident distinct straight projected edges, and exact vertex/edge equivariance under the complete symmetry group. It deliberately does not claim global absence of crossings between arbitrary nonincident periodic edge images; that finite periodic spatial certificate belongs to Stage 8B. Distorted or finite-temperature trajectory coordinates remain downstream geometry and do not redefine the authoritative tiling reference.

Revision 26′s certification and persistence boundary remains normative. The implemented stack distinguishes

$$\text{scientific result} \neq \text{derived index} \neq \text{search workspace} \neq \text{verification certificate}\ .$$

`PeriodicNetSymmetry` stores only the finite net automorphism group and its translation cocycle. Primitive-ring actions remain in the separately catalog-bound `PrimitiveRingSymmetryIndex`. `RingStrengthResult` remains compact and independently replay-verifiable, while `RingStrengthSearchWorkspace` remains transient. Exact finite $\mathrm{GF}(2)$ elimination retains explicit matrix and provenance resource guards.

Revision 25′s bounded strong-ring mathematics remains normative. In the depth-one Na-LTA domain, all 36 four-rings are boundedly strong, the 40 six-rings split into 24 weak and 16 boundedly strong rings, and all six eight-rings are boundedly strong. These remain finite-domain topological results, not natural-tiling face assignments.

Revision 24′s exact automatic symmetry discovery remains normative. The eligible unlabeled Na-LTA T-net has 96 representatives modulo translations, one vertex orbit, three edge orbits, and five primitive-ring orbits of sizes 6, 12, 16, 24, and 24. The normalized-operation composition cocycle remains required for correct action on absolute lifted placements.

Revision 22′s view-bound automorphism validation and revision 21′s `PeriodicNetView` semantics remain normative. The first net-view backend is a signature projection only: it preserves the exact framework vertex and edge orbits while defining which decorations symmetry must preserve.

Revision 20′s source-safe ring identities, shared periodic arithmetic, `CycleParameterization`, and removal of `max_component_count` from the mathematical strength domain remain normative. Primitive-ring enumeration, classification, catalog digests, and physical-edge support semantics are unchanged by this consolidation.

The revision-16 geometric commitments remain normative:

1. the periodic spatial backend is a **query-agnostic geometric broad phase** that generates conservative `(object_i, object_j, image_shift)` candidates from continuous lifted support bounds; distance, intersection, penetration, linking, containment, and volume overlap remain consumer predicates;
2. multi-bin occupancy retains periodic image labels, and multi-image candidate caches reuse the existing deformation-aware Verlet **validity theorem/kernel** without inheriting the atomic backend's unique-image/MIC assumptions;
3. scientific `FacePlacement` identity is separated from auxiliary `FaceEmbeddingWitness` triangulations, just as `PeriodicCellComplex` is separated from `PeriodicPartitionCertificate`;
4. ring catenation uses rigorous certificate semantics: nonzero algebraic ring–spanning-surface intersection certifies linking, while intersection of two particular spanning-disk witnesses means only that those witnesses are incompatible; disjoint embedded disk witnesses certify unlinking for the disk-bounding two-component case;
5. tile overlap is defined by **interior-volume** overlap, not by ordinary boundary contact; prescribed shared faces/edges/vertices are allowed, while improper crossings or containment overlap are rejected; and
6. properness is a property of the scientific face/tile complex. Auxiliary disk triangulations and partition meshes need only certify the scientific structure and are not themselves required to have identical symmetry combinatorics.

Stage 5 still introduces no second scientific ring catalog and no premature public `PeriodicEdgeChain` contract. A formal oriented chain complex appears only with the actual periodic cell complex, where its source graph, integer coefficients, and boundary operators are fully defined.

The translated-placement, automorphism-induced occurrence-mapping, and exact finite modulo-two cancellation prototypes are implemented and gated, and their shared Stage-5 infrastructure has been cleaned and frozen. Revision 22 advances the P2 prototype into a view-bound validation layer: `periodic_ring_action.py` now enforces `PeriodicNetView` signatures and source identity while retaining the exact occurrence-level ring action.

An untruncated `SHORTEST_PATH_PAIRS` result remains the sole scientific catalog of local, zero-winding primitive-ring translation orbits in the requested size interval; `PrimitiveRingIndex` remains a transient acceleration/identity layer. The current symmetry layer validates explicit operations and assembles the exact finite subgroup they generate modulo translations. Automatic discovery of a complete generator set and bounded strength are implemented. Natural-tiling orchestration and cages remain downstream commitments.

This is a **high-level design manual**, not a module specification. It preserves scientific definitions, correctness claims, module boundaries, provenance, and implementation order before detailed APIs are finalized.

The manual should answer eight questions during implementation:

1. What scientific responsibility belongs to each module?
2. Which periodic graph is authoritative for symmetry and tiling?
3. Which representation is authoritative at each layer?
4. Which results are complete, bounded, unresolved, conditional, or heuristic?
5. How are topology, embedding, tiling, and chemical interpretation separated?
6. Which exact group acts on vertices, edges, rings, embedded faces, and tiles?
7. Which external algorithms are adopted or adapted?
8. Which features are intentionally deferred?

Theoretical background

From an atomic configuration to a graph

An atomistic frame is first a geometric object: atom i has a species label and position $\mathbf{r}_i$ in a periodic cell. Ring analysis begins only after a **connectivity model** converts that geometry into a graph

$$G_{\mathrm{a}} = (V_{\mathrm{a}}, E_{\mathrm{a}}).$$

The vertex set identifies the atoms admitted to the connectivity problem. An edge asserts a discrete relation under an explicit model: distance threshold, hysteresis, reference connectivity, ensemble consensus, or a user-supplied classifier. The graph is therefore not uniquely determined by the coordinates. Different scientifically reasonable connectivity models can produce different rings from the same frame.

This architecture treats connectivity as an inferred scientific state with provenance, not as an intrinsic property silently read from a coordinate file. Radial neighbors, chemical bonds, projected framework edges, and transport connections are related concepts, but they are not interchangeable.

Periodic nets and finite quotient graphs

An infinite periodic framework can be represented by a finite graph whose edges carry lattice translations. This labelled-quotient-graph or vector-method view is established in the crystallographic-net literature [1, 2].

Let i and j denote vertices in one reference cell. A periodic edge is written

$$e = (i, j, \mathbf{m}), \qquad \mathbf{m} \in \mathbb{Z}^3,$$

meaning that i in image $\mathbf{n}$ is connected to j in image $\mathbf{n} + \mathbf{m}$. The infinite lifted graph has vertices

$$(i, \mathbf{n}), \qquad \mathbf{n} \in \mathbb{Z}^3,$$

and the finite labelled graph contains enough information to regenerate every periodic copy.

Translation labels depend on the chosen image representative for each finite vertex. If vertex i is reassigned by an integer image vector $\mathbf{g}_i$, then an edge label transforms as

$$\mathbf{m}'_{ij} = \mathbf{m}_{ij} + \mathbf{g}_j - \mathbf{g}_i.$$

This **periodic gauge freedom** changes the finite labels but not the infinite net. Canonicalization must therefore choose a deterministic gauge before exact comparison or hashing. Gauge normalization is representational bookkeeping; it does not alter the topology.

Framework projection and decorated edges

Many atomic frameworks contain chemically meaningful linker atoms between the vertices used for topological analysis. The projected framework graph replaces an atomic path

$$A - L_1 - L_2 - \cdots - L_k - B$$

by one decorated edge between endpoint vertices A and B. The decoration retains the ordered linker sequence, atomic path, and periodic translations. The projected graph is a quotient of the atomic connectivity graph under a specified vertex/linker mapping; it is not a generic geometric simplification.

This distinction is especially important in mixed-linker or directionally decorated frameworks. Reversing the whole path produces

$$B - L_k - \cdots - L_2 - L_1 - A,$$

but independently reversing only the endpoint order or only the linker sequence changes the decorated edge. The authoritative adjacency may remain undirected while traversal retains an orientation sign and a reversible ordered path.

Cycles, local rings, and primitive rings

A graph-theoretic cycle is a closed path with no repeated internal vertex. In a periodic labelled graph, a closed walk also carries a lattice winding

$$\mathbf{w}(C) = \sum_{e \in C} \epsilon_e \mathbf{m}_e,$$

where $\epsilon_e = \pm 1$ records traversal orientation. A **local ring** closes in the infinite lift and therefore satisfies

$$\mathbf{w}(C) = \mathbf{0}.$$

A nonzero-winding quotient walk is not a finite cycle in the lifted graph.

The architecture uses the primitive-ring definition studied by Goetzke and Klein and by Yuan and Cormack [5, 6]: a cycle is primitive when it cannot be written as the symmetric-difference sum of two smaller cycles. For the unweighted lifted graph, this is equivalent to the **no-strict-shortcut** criterion: no pair of ring vertices admits a graph path shorter than the shorter cycle arc between them.

For a cycle C and vertices $u, v \in C$, let $d_C(u, v)$ be the shorter cycle-arc length. Then

$$C \text{ is primitive} \quad \Leftrightarrow \quad d_{\tilde{G}}(u, v) = d_C(u, v) \quad \text{for all} \quad u, v \in C.$$

Only maximal half-cycle pairs must be checked. If

$$r = \left\lfloor \frac{|C|}{2} \right\rfloor,$$

every shorter cycle arc is a subpath of a length-r arc, and every subpath of a shortest path is shortest.

The implemented shortest-path-pair algorithm is complete for bounded periodic rings, not merely for a finite quotient approximation. Let $G = \tilde{G}/\Lambda$ be the finite quotient and

$$S = \{(u, \mathbf{0}) : u \in V(G)\}.$$

Every finite lifted cycle may be translated so that one of its vertices lies in S. Every cycle of size at most K then lies within distance $\lfloor K/2 \rfloor$ of S. Moreover, every strict-shortcut witness relevant to such a cycle lies within distance K of S. The finite induced graph

$$H_K = \tilde{G}[\{x : d_{\tilde{G}}(x, S) \leq K\}]$$

therefore contains a representative of every bounded ring orbit and every witness that can change its primitive classification. Since $\tilde{G}$ is locally finite and S is finite, H_K is finite. Finite-graph primitive-ring results consequently apply to the bounded periodic problem.

This proof is an original `mdstats` derivation from the periodic covering-graph model. It is stated fully in the `primitive_ring.py` architecture section and in `primitive_ring_spec.md`.

Topology, embedding, tiling, and chemical interpretation

Natural tiling is most cleanly formulated as construction of a periodic cell complex. In the quotient of Euclidean space by the translation lattice,

$$T^3 = \mathbb{R}^3/\Lambda,$$

the scientific objects are

Dimension	Object
0	framework vertex orbit
1	framework edge orbit
2	selected embedded ring-face orbit
3	tile orbit

with boundary maps

$$C_3 \xrightarrow{\partial_3} C_2 \xrightarrow{\partial_2} C_1 \xrightarrow{\partial_1} C_0,$$

satisfying

$$\partial_2 \partial_3 = 0, \qquad \partial_1 \partial_2 = 0.$$

This viewpoint fixes the responsibility of each scientific layer:

1. `FrameworkTopology` defines the periodic 1-skeleton.
2. `PrimitiveRingCatalog` supplies a complete bounded family of candidate 2-cell boundaries.
3. `PeriodicNetView` defines the exact periodic multigraph used for symmetry and properness.
4. Stage-5 views expose stable-key ring placements, explicit boundary parametrizations, and exact physical edge support without creating another scientific catalog.
5. `PeriodicNetSymmetryCatalog` defines the multigraph automorphism action.
6. `RingStrengthCatalog` classifies bounded strong-ring status in an explicit finite domain.
7. `FaceCandidateCatalog` separates scientific face placements from embedded witness surfaces and records compatibility/linking constraints.
8. `PeriodicCellComplex` represents and partition-certifies candidate 3-cells.
9. `NaturalTilingCatalog` records the certified or conditional result selected by the natural-tiling rules.
10. Ring-site and cage analyses add frame-dependent geometry and chemical meaning.

A primitive ring is not automatically a face. A strong embedded ring is not necessarily essential. Essentiality is defined only after a valid natural tiling has been accepted. Likewise, a natural tile is not automatically a physically accessible cage.

The natural-tiling rules of Blatov, Delgado-Friedrichs, O'Keeffe, and Proserpio select a proper tiling carried by the net and use locally strong ring faces [9]. The zeolite application and LTA validation targets follow Anurova et al. [10]. The 2023 review further distinguishes strong rings from rings that are essential to a selected tiling [11].

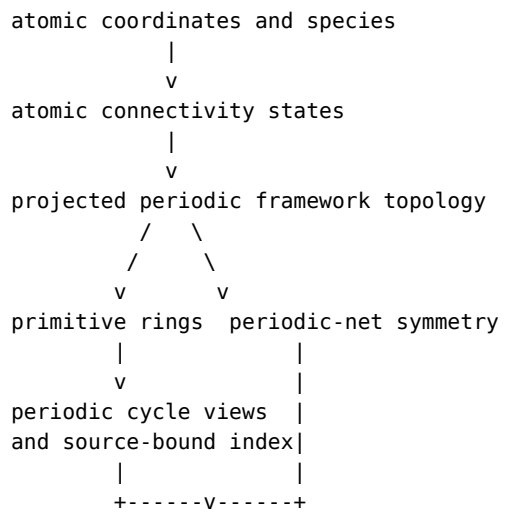
Design goals

The architecture should support zeolite analysis immediately while remaining general enough for other periodic, distorted, partially broken, and disconnected frameworks.

The principal goals are:

- preserve exact periodic graph identity independently of Cartesian distortion;
- represent arbitrary framework vertex species, linker paths, parallel edges, and lattice translations;
- compress repeated connectivity and topology states across trajectories and ensembles;
- make bounded primitive-ring completeness explicit and machine-checkable;
- centralize shared periodic-graph operations without replacing established public data models;
- preserve exact physical lifted-edge support before adjacency or face semantics;
- classify symmetry from periodic graph automorphisms, not instantaneous space group fits;
- distinguish primitive, strong, locally strong, and essential rings;
- require embedded-face, cell-complex, and properness certificates before promoting objects downstream;
- preserve bounded or unresolved status when a proof is unavailable;
- keep topological tiles separate from frame-dependent geometry and guest accessibility; and
- remain independently testable and replaceable at every stage.

The central dependency graph is



```
               |
               v
bounded strong-ring classification
               |
               v
embedded face candidates
               |
               v
natural-tiling search and periodic cell construction
               |
               v
natural tiling
```

Frame-dependent interpretation branches after persistent topology:

```
primitive rings -> ring geometry -> ring sites
natural tiling  -> tile geometry -> cages and portals
```

The phrase **natural tiling** is reserved for a validated periodic cell complex that also passes properness and the natural-selection rules. A closed or space-filling face complex without that certificate remains a periodic tiling.

Core separation of concerns

Frame semantics and topology semantics

`AtomisticFrameCollection` already distinguishes time-ordered trajectories from unordered structural ensembles:

```
FrameSemantics.TRAJECTORY
FrameSemantics.ENSEMBLE
```

This distinction answers only:

> Are the frames physically time ordered and dynamically continuous?

It does not imply anything about chemical connectivity. Topology consistency is an independent analysis result.

A trajectory may retain one topology, undergo one irreversible event, alternate between several topologies, or change almost every frame. An ensemble may contain independent configurations of one framework topology or several distinct topology classes.

The base collection must therefore remain topology-agnostic.

Geometric neighborhood and atomic connectivity

The architecture must distinguish two scientifically different relations.

A **geometric neighborhood** is an instantaneous radial predicate such as

$$j \in \mathcal{N}_i(r_c) \Leftrightarrow r_{ij} < r_c.$$

This definition is simple, local, and appropriate for RDF-derived coordination, neighbor-angle distributions, liquids, and thermally disordered structures. It does not necessarily assert a persistent chemical bond.

An **atomic connectivity model** produces a discrete decorated edge set

$$E_f = \{(i, j, \mathbf{m})\}$$

for frame f, where $\mathbf{m}$ records the periodic image of the connected atom. Connectivity may be inferred from distance thresholds, trajectory hysteresis, a reference graph, an ensemble consensus rule, explicit bonds, bond order, or a user-supplied classifier.

The distinction is scientifically important. A Na-O radial neighbor used for coordination analysis is not automatically a persistent chemical bond. Conversely, a framework edge used for ring topology must be defined by an explicit connectivity model whose provenance is recorded.

The dependency is therefore

```
coordinates
    |
    v
geometric pair data from _neighbors.py
    |
    v
AtomicConnectivityDefinition
    |
    v
AtomicConnectivityResult
    |
    v
FrameworkMapping and projected topology
```

The base collection remains unaware of both neighborhood and topology semantics.

Connectivity scope and framework participation

Atomic connectivity and framework membership are related but distinct decisions. A radial or hysteretic connectivity rule may identify contacts involving atoms that are not part of the structural framework of interest. In an aluminosilicate, for example, Na, Li, and K may be close to several framework oxygen atoms and may be useful in coordination or adsorption analysis, but they must not become framework vertices or linker atoms merely because a distance threshold is satisfied.

The connectivity layer should therefore support an explicit **connectivity scope**:

```
ConnectivityScope
|-- included species or atom indices
|-- excluded species or atom indices
`-- deterministic precedence and provenance
```

The scope answers:

> Which atoms are eligible to participate in this atomic-connectivity calculation?

It does not decide their role in the projected framework. Framework participation is assigned later by `FrameworkMapping`, using explicit atom roles:

```
FrameworkAtomRole.VERTEX
FrameworkAtomRole.LINKER
FrameworkAtomRole.SPECTATOR
FrameworkAtomRole.EXCLUDED
```

`VERTEX` atoms remain as nodes in the projected graph. `LINKER` atoms may occur inside contracted paths. `SPECTATOR` atoms are intentionally outside the framework but remain scientifically relevant for later site, adsorption, occupancy, and transport analysis. `EXCLUDED` atoms are ignored by the current framework analysis.

Species defaults must be overridable by explicit atom-index selections. This is required when one species occupies several structural roles, such as framework and extra-framework Al, bridging and terminal O, or framework-bound and mobile metal atoms.

The two layers serve different purposes:

```
ConnectivityScope:
    controls which atomic edges are evaluated

FrameworkMapping roles:
    control how eligible atoms participate in framework projection
```

A framework-scoped calculation may include only Si, Al, and O for efficiency. A broader calculation may retain Na-O or K-O connectivity while marking alkali ions as spectators, allowing the same connectivity data to support later guest analysis without contaminating framework topology.

Persistent identity scope and dynamic region membership

`ConnectivityScope` is a persistent atom-identity selection. It answers which canonical atoms are eligible to participate in one connectivity calculation. It must not change merely because an atom moves across a slab boundary, leaves a solid, or enters a liquid.

This rule is essential for reactive and heterogeneous systems. Consider an oxygen atom that belongs to the original zeolite framework but later breaks away and moves into a molten salt. The topology analysis must retain that oxygen in the candidate framework population so that its lost bonds, changed connectivity state, and destroyed rings remain visible. Dynamically removing the oxygen from scope would erase the event rather than characterize it.

The architecture therefore distinguishes three independent properties:

```
original atom identity or region assignment
current geometric or phase region
current structural attachment or connectivity
```

For one atom these may disagree. A detached framework oxygen may be

```
original region:                 zeolite framework
current geometric region:        liquid
connected to framework backbone: no
```

A mobile Na ion entering a cage may instead be

```
original region:                 liquid or spectator population
current geometric region:        zeolite interior
connected to framework backbone: no
```

Dynamic spatial and phase classification belongs to a separate future module:

```
region_membership.py
```

That module may consume coordinates, reference atom sets, connectivity catalogs, and topology catalogs, but it must not redefine the authoritative connectivity scope. Its responsibilities may grow through three levels:

1. fixed geometric regions, such as Cartesian or fractional slabs and boxes;
2. reference-aligned regions that follow rigid translation or rotation of anchor atoms;
3. topology-aware dynamic regions that identify anchor-connected solid backbones, detached fragments, moving interfaces, and transfer between materials.

For a slab with known interface normal $\hat{\mathbf{n}}$, a simple geometric coordinate is

$$z_i = \mathbf{r}_i \cdot \hat{\mathbf{n}}.$$

A fixed slab region might classify atoms using

$$a < z_i < b.$$

If the slab drifts, the region should instead be evaluated in a reference-aligned coordinate system. Translation-only alignment may use the anchor displacement

$$\Delta \mathbf{R}_t = \mathbf{r}^{\mathrm{COM}}_{\mathrm{anchor}}(t) - \mathbf{r}^{\mathrm{COM}}_{\mathrm{anchor}}(0).$$

A more advanced topology-aware classifier may define the attached solid backbone as the connected component containing designated seed atoms. Original solid atoms outside that component are detached even if they remain geometrically close to the surface.

The governing principle is

$$\boxed{\text{persistent identity scope} + \text{connectivity state} + \text{dynamic region membership}}$$

These layers must remain independently inspectable and reportable.

Relationship to RDF, coordination, and angle statistics

RDF remains a pair-density statistic and must continue to use radial pair distances. It must not be restricted to connectivity edges unless a separately named conditional distribution is requested.

Coordination and angle analysis may eventually accept connectivity-defined relations, but their existing radial definitions remain first-class and explicit:

$$N_i^{AB}(r_c) = \sum_{j \in B} \mathbf{1}(r_{ij} < r_c),$$

and

$$P^{(r_c)}_{A-B-C}(\theta)$$

are different observables from graph degree and connectivity-defined bond angles. The package must never silently replace one definition by the other.

A future implementation may expose separate functions or an explicit `neighbor_definition` argument, but every result must record whether its triplets or counts came from a radial rule or a connectivity model. Scientific reports must be able to state the definition and thresholds unambiguously.

Topology consistency categories

The topology layer should classify an analyzed collection as one of:

```
TopologyConsistency.UNDEFINED
TopologyConsistency.UNIFORM
TopologyConsistency.PARTITIONED
TopologyConsistency.PER_FRAME
```

`UNDEFINED` means that no topology claim has been made.

`UNIFORM` means every analyzed frame maps to one canonical topology.

`PARTITIONED` means multiple topology classes are identified, reconciled, and reused through one topology catalog. For a trajectory, contiguous segments are also recorded. For an ensemble, frames are grouped by topology class without assigning physical significance to frame order. The category does not depend on an arbitrary threshold for what counts as a "small" number of classes.

`PER_FRAME` means that topology is intentionally treated as independent for each frame, without promising class reconciliation or persistent topology identity. Repeated fingerprints may still be cached internally.

The category is descriptive rather than a property stored permanently on `AtomisticFrameCollection`.

Topology classes and trajectory segments

Topology classes and contiguous segments must not be conflated.

For a trajectory with topology sequence

```
A A A B B A A
```

there are two topology classes, `A` and `B`, but three ordered segments:

```
A | B | A
```

A topology catalog should therefore retain:

- unique topology objects;
- a frame-to-topology mapping;
- contiguous segments for trajectories;
- unordered frame groups for ensembles;
- transitions and graph differences where meaningful.

This separation enables compressed storage and analysis of recurring topology states.

General framework model

Atomic connectivity source

Framework projection should consume an atomic connectivity graph rather than hard-code one permanent definition of a bond. The first implementation may use pair-distance rules, but the architecture must allow alternative sources:

- fixed radial cutoffs from `PairCutoffRegistry`;
- two-threshold trajectory hysteresis;
- a fixed reference edge set with validation tolerances;
- an ensemble-consensus edge model;
- explicit user-supplied bonds;
- file-format topology;
- bond-order or user-callback classifiers.

The connectivity rule is immutable scientific provenance. Its evaluated edge sets and state catalog are separate result objects. A connectivity definition may also carry a `ConnectivityScope` that limits eligible atoms without assigning framework roles.

Atomic graph and projected framework graph

The chemically explicit atomic graph is

$$G_{\mathrm{atom}} = (V_{\mathrm{atom}}, E_{\mathrm{bond}}),$$

where atoms are vertices and chemically defined bonds are edges.

The user selects retained framework vertices

$$V_F \subseteq V_{\mathrm{atom}},$$

and defines how atomic paths are projected into framework edges. A projected edge between retained vertices v_i and v_j may represent

$$P_{ij} = (v_i, \ell_1, \ell_2, \ldots, \ell_m, v_j),$$

where the internal atoms ℓ_k form a linker path.

Examples include:

Physical connection	Atomic path represented by one framework edge
Direct bond	$A - B$
Oxygen bridge	$A - O - B$
Sulfur bridge	$A - S - B$
Peroxide bridge	$A - O - O - B$
Persulfide bridge	$A - S - S - B$
Longer linker	$A - L_1 - \cdots - L_m - B$

For an aluminosilicate framework,

$$\mathrm{T} - \mathrm{O} - \mathrm{T} \rightarrow \mathrm{T} - \mathrm{T},$$

with Si and Al retained as framework vertices and each bridging oxygen retained as the decorated atomic path of a projected edge. Alkali ions such as Na, Li, and K are normally assigned `SPECTATOR` roles. Their proximity to oxygen may remain available for separate analysis, but they cannot become projected vertices or internal linker atoms unless the mapping explicitly says otherwise.

Undirected adjacency and orientation-aware path decoration

Atomic bonds are reversible relations, so a contracted framework connection is normally one undirected adjacency even when the ordered linker chemistry is asymmetric. For a path

$$P_{uv} = (u, \ell_1, ..., \ell_m, v),$$

the reverse traversal is

$$P_{vu} = P_{uv}^{-1} = (v, \ell_m, ..., \ell_1, u).$$

The two traversals represent one physical projected edge. However, endpoint species and linker order must be canonicalized as one complete path signature. For example,

$$\boxed{A - O - S - B \equiv B - S - O - A}$$

while

$$\boxed{A - O - S - B \not\equiv A - S - O - B}\,.$$

The authoritative framework graph therefore remains an undirected periodic multigraph, but every edge stores one canonical ordered atomic path and supports a derived traversal orientation $\eta \in \{+1, -1\}$. Reversing η reverses the path and linker sequence and negates all directed periodic translations. This orientation is part of edge decoration and ring traversal; it is not an intrinsic reaction or transport arrow.

Decorated periodic multigraph

The projected framework graph should be a periodic decorated multigraph:

$$G_F = (V_F, E_F).$$

Each projected edge must retain enough information to reconstruct the complete atomic connection in either traversal orientation:

- canonical endpoint framework-vertex atom indices;
- all ordered internal linker atom indices and species;
- the complete canonical atomic path;
- periodic image shifts along the path;
- the target image relative to the canonical source;
- a deterministic canonical edge-path key;
- a derived reverse traversal that reverses the complete path and negates all directed translations.

Endpoint species and linker order are one coupled rule signature. They must not be canonicalized independently.

A multigraph is required because distinct atomic paths or periodic edge instances may connect the same canonical pair of framework vertices.

Unrestricted framework degree

The graph representation must not assume tetrahedral coordination or any fixed vertex degree. Zeolite-specific statements such as

$$\deg(\mathrm{Si}) = 4$$

belong to optional validation rules, not to graph construction.

This permits later use with:

- amorphous alumina containing four-, five-, and six-coordinate Al;
- chalcogenide networks;
- phosphates and sulfates;
- metal-organic frameworks;
- defective or reactive frameworks;

- mixed-species networks with material-specific degree rules.

Bounded linker discovery

The data model should permit arbitrarily long linker paths, but automatic path search must be bounded to avoid combinatorial growth.

A framework mapping should eventually specify items such as:

- retained vertex selections;
- allowed linker selections;
- minimum and maximum number of internal atoms;
- whether internal linkers must form a simple path;
- whether internal linker atoms must have degree two within the selected mapping;
- optional user-defined linker recognizers;
- explicit enforcement that every internal path atom is assigned the `LINKER` role and that spectators or excluded atoms cannot be traversed.

The first implementation may support direct bonds and short simple linker chains while preserving data structures that do not prevent later generalization.

Mapping-dependent ring size

A ring in the projected graph has at least two useful sizes.

The projected framework size is

$$n_F = \text{number of retained framework vertices}\ .$$

The atomic perimeter length is

$$n_{\text{atomic}} = \sum_{e \in C} (\mid P_e \mid - 1),$$

where $|P_e| - 1$ is the number of atomic bonds represented by projected edge e.

For an ordinary zeolite n-ring,

$$n_F = n, \qquad n_{\text{atomic}} = 2n.$$

Ring statistics must always state which graph and size convention they use.

Atomic connectivity architecture

Connectivity scope

A connectivity definition may be evaluated over all atoms or over a restricted scope. Conceptually:

```
ConnectivityScope(
    included_species=None,
    excluded_species=(),
    included_atom_indices=None,
    excluded_atom_indices=(),
)
```

Explicit exclusions take precedence over inclusions. Atom-index rules take precedence over species defaults. The resolved scope must be deterministic and recorded in provenance.

Scope is a computational and scientific boundary. Excluding Na from the scope means that no Na-containing connectivity edge exists in the result. Retaining Na in the scope but assigning it `SPECTATOR` status later means that Na-containing edges exist but cannot alter framework topology.

The first implementation should permit a common scope on all supported connectivity definitions. The resolved scope is fixed across the analyzed frames and is based on persistent atom identity, not current position or phase. Hysteretic and reference

rules must apply only to edges whose endpoints remain inside that fixed scope. Dynamic slab, interface, or phase labels are evaluated by `region_membership.py` and must not silently add or remove atoms from the connectivity graph.

Definition objects and evaluated results

The package should separate an immutable connectivity rule from its evaluated output.

Conceptual definition types include:

```
AtomicConnectivityDefinition
|-- DistanceConnectivity
|-- HystereticDistanceConnectivity
|-- ReferenceConnectivity
|-- ConsensusConnectivity
`-- ExplicitConnectivity
```

The evaluated result should preserve canonical atomic edge identities, periodic image shifts, per-frame connectivity-state assignment, transitions where meaningful, and complete provenance.

Distance connectivity

A frame-local radial connectivity rule accepts an edge when

$$r_{ij} < r_{\mathrm{cut}}^{AB}.$$

It is deterministic and valid for either trajectories or ensembles, but it may flicker when thermal motion crosses one threshold.

Hysteretic trajectory connectivity

For a time-ordered trajectory, a two-cutoff model may use

$$r_{\mathrm{form}}^{AB} < r_{\mathrm{break}}^{AB}.$$

An absent edge forms only when

$$r_{ij}(t) < r_{\mathrm{form}}^{AB},$$

while an existing edge is retained until

$$r_{ij}(t) \geq r_{\mathrm{break}}^{AB}.$$

This suppresses artificial edge flicker near one cutoff. Because the rule depends on the previous state, it must never be applied to an ensemble in arbitrary stored order.

Reference connectivity for ensembles

Independent configurations of one intended framework may be compared with a reference edge set. Reference edges may use a retention tolerance, while unexpected edges require a stricter formation condition. This provides robust classification without pretending that ensemble frames are time ordered.

Ensemble-consensus connectivity

For an ensemble, a persistent edge may be inferred from its occupancy

$$p_{ij} = \frac{1}{F}\sum_{f=1}^{F}\left[\mathbb{1}_{ij}^{(f)} < r_{\mathrm{candidate}}\right].$$

An edge is accepted when $p_{ij} \geq p_{\min}$. This model is useful for deriving a consensus topology, but its result depends on the sampled ensemble and must report both the candidate cutoff and occupancy threshold.

Connectivity state compression

Atomic connectivity may itself be compressed into reusable states before framework projection. A long trajectory with one bond-breaking event should not store one full edge graph per frame.

The connectivity layer may therefore retain:

- unique canonical connectivity states;
- frame-to-state IDs;
- contiguous trajectory segments;
- ensemble frame groups;
- added and removed atomic edges;
- candidate and confirmed changes;
- the connectivity definition and schema version.

Framework topology then projects each unique connectivity state rather than every frame independently.

Scientific naming

The package and documentation should use precise names:

- **radial coordination** for counts under a distance cutoff;
- **connectivity degree** for graph degree under a connectivity model;
- **neighbor-angle distribution** when radial neighbors define triplets;
- **bond-angle distribution** when connectivity edges define triplets.

Both forms may be useful, but their definitions and provenance must remain visible.

Module architecture

The accepted analysis stack is

```
mdstats/analysis/
|-- atomic_connectivity.py
|-- framework_topology.py
|-- topology_catalog.py
|-- region_membership.py
|-- primitive_ring.py
|-- periodic_net_view.py
|-- periodic_barycentric.py
|-- net_symmetry.py
|-- net_symmetry_discovery.py
|-- periodic_net_embedding.py
|-- periodic_cycle.py
|-- primitive_ring_index.py
|-- primitive_ring_symmetry.py
|-- ring_strength.py
|-- face_candidates.py
|-- periodic_cell_complex.py
|-- natural_tiling.py
|-- ring_geometry.py
|-- ring_site.py
|-- tiling_geometry.py
`-- cage.py
```

`PeriodicNetView` is the explicit combinatorial boundary between the chemically decorated `FrameworkTopology` and symmetry/natural-tiling operations. In the first backend it preserves the exact projected vertex and edge orbit sets and changes only the signatures that automorphisms must preserve. Ignoring an attribute permits exchange under automorphism; it never removes, contracts, or merges an edge.

`PeriodicNetEmbedding` is a separate object. It assigns one validated, symmetry-compatible Euclidean embedding to that exact net view. Natural-tiling face and partition topology is constructed on this reference embedding; distorted MD frames are mapped onto the accepted topology only for downstream geometry.

Stage-5 infrastructure is intentionally source-bound and lightweight. The persistent scientific result remains `PrimitiveRingCatalog`. Translated ring placements, boundary parametrizations, inverse incidence, and exact lifted-edge supports are derived views or caches rather than a second catalog.

Shared private helpers are extracted only after the symmetry-mapping and strength-placement prototypes reveal concrete reuse. A possible private module

`mdstats/analysis/_periodic_graph.py`

may own exact lattice-shift arithmetic, relative-image conversion, and physical edge-instance anchors. It must not erase the semantic distinction between `LiftedAtomRef` and `LiftedVertexRef`, replace the public graph records, or absorb object-specific reversal and canonicalization.

Existing geometric infrastructure remains below the scientific modules:

```
cutoffs.py
_neighbors.py
AtomisticFrameCollection
```

`_neighbors.py` supplies candidate pairs, minimum-image vectors, distances, safe-radius checks, and blocked evaluation. It does not define bonds, rings, faces, or tiles.

Additional private helpers are introduced only when required by a concrete specification. Planned geometric broad-phase infrastructure includes one private module, initially

`_periodic_spatial.py`

for complete periodic translation stencils, automatic extended-object linked-cell grids, multi-bin occupancy, candidate object/image pairs, and a thin adapter to the shared deformation-aware Verlet validity kernel. This module is Euclidean/geometric only; graph reachability remains in graph modules.

Other private helpers include

```
_bounded_paths.py
_gf2.py
_robust_geometry.py
_surface_mesh.py
_periodic_partition.py
```

An external algorithm adapted in a private helper must be cited in the module specification and implementation comments. A helper should not be created merely to anticipate hypothetical reuse.

`atomic_connectivity.py`

Responsibility

`atomic_connectivity.py` converts geometric pair information or explicit bond data into deterministic decorated atomic connectivity states.

It is the authoritative layer for:

- what one atomic connectivity edge means;
- how connectivity is evaluated across trajectories or ensembles;
- formation, retention, and breaking criteria;
- canonical atomic edge identity and periodic image shifts;
- connectivity-state fingerprints and transitions;
- provenance of the connectivity definition.

Primary inputs

Conceptually, the module consumes:

- one `AtomisticFrameCollection`;
- an immutable `AtomicConnectivityDefinition`;
- species-pair selections and any cutoff registry;
- an optional immutable `ConnectivityScope`;
- optional reference edges or explicit connectivity;
- frame selection and robustness policy.

Primary outputs

Conceptual output objects include:

```
AtomicConnectivityDefinition
AtomicConnectivityState
AtomicConnectivityResult
AtomicConnectivityTransition
ConnectivityValidationResult
```

The exact implemented API is defined in `docs/specs/analysis/atomic_connectivity_spec.md`.

Owned operations

The module owns:

- resolving deterministic included and excluded atom scopes;
- evaluating distance, hysteretic, reference, consensus, or explicit rules;
- canonicalizing periodic atomic edges;
- grouping repeated connectivity states;
- trajectory run-length segmentation;
- ensemble grouping without temporal assumptions;
- distinguishing geometric violations, candidate changes, and confirmed changes;
- reporting added and removed atomic edges;
- stable schema-versioned fingerprints.

Explicit non-responsibilities

It must not:

- project atomic paths into framework edges;
- search for primitive rings;
- redefine RDF;
- silently change radial coordination or neighbor-angle semantics;
- decide whether an eligible atom is a framework vertex, linker, spectator, or excluded framework participant.

`framework_topology.py`

Responsibility

`framework_topology.py` converts one atomic frame into one deterministic periodic decorated framework topology.

It is the authoritative layer for the meaning of:

- framework vertex;
- linker path selected from an input atomic connectivity graph;
- projected framework edge;
- periodic edge translation;
- single-frame topology fingerprint;
- material-specific topology validation.

Primary inputs

Conceptually, the module consumes:

- one `AtomisticFrameCollection`;
- one frame index;
- a framework mapping;
- one immutable `AtomicConnectivityState` or compatible connectivity result;
- optional validation rules.

The module should not rebuild bonds independently once the connectivity layer exists. `_neighbors.py` remains the lower geometric kernel used by connectivity models.

Primary outputs

The module should eventually define objects equivalent in role to:

`FrameworkAtomRole`
`FrameworkMapping`
`FrameworkEdgePath`
`FrameworkTopology`
`FrameworkValidationOptions`
`FrameworkValidationResult`

The exact field layout will be specified separately.

Owned operations

The module owns:

- resolving framework atom roles from species defaults and atom-level overrides;
- selecting retained framework vertices;
- excluding spectators and excluded atoms from projected paths;
- consuming and validating the required atomic connectivity graph;
- discovering bounded linker paths;
- contracting accepted paths into decorated projected edges;
- preserving periodic image translations;
- deterministic vertex and edge ordering;
- complete path-signature matching modulo whole-path reversal;
- orientation-aware traversal views for asymmetric linker paths;
- detecting parallel edge paths;
- calculating a topology fingerprint;
- optional degree, linker, component, and mapping validation;
- periodic gauge normalization;
- exact labeled-topology canonicalization;
- schema-versioned stable serialization and digest generation.

Explicit non-responsibilities

It must not:

- compare multiple frames;
- classify topology consistency;
- search for rings;
- calculate ring centers or normals;
- classify sites or cages.

Important invariants

- Atom indices refer to canonical indices in the source collection.
- Distinct periodic or chemically distinct edge paths remain distinct.
- `A-O-S-B` is equivalent to `B-S-O-A` but distinct from `A-S-O-B`.
- Canonical undirected edge identity and oriented path traversal are separate interfaces.
- Graph construction does not impose material-specific coordination unless requested by validation.
- Only `VERTEX` atoms may become projected nodes, and only `LINKER` atoms may occur as internal path atoms.
- `SPECTATOR` and `EXCLUDED` atoms cannot alter projected connectivity.
- Atom-level role overrides take precedence over species-level defaults and must be part of mapping provenance and canonical identity.
- The topology fingerprint depends on connectivity, mapping, species, periodic edge data, and a canonical-schema version, not on arbitrary graph traversal order or Cartesian orientation.
- Topology equality within a compatible atom-indexed collection is exact equality of canonical decorated edge records, not general unlabeled graph isomorphism.
- Disconnected and partially broken frameworks are valid representations; connectedness is an optional validation rule.

- Identity-bearing topology objects are immutable after construction.
- Periodic edge translations are normalized to a deterministic gauge before they enter canonical keys or fingerprints.

`topology_catalog.py`

Responsibility

`topology_catalog.py` is the implemented Stage 3 layer for exact framework topology classification across selected frames. It projects each referenced `AtomicConnectivityState` once through one fixed `FrameworkMapping`, reconciles exact Stage 2 structural keys, and records topology classes, frame groups, trajectory segments, and transition-local edge differences.

It is authoritative for:

- `TopologyConsistency.UNIFORM`, `PARTITIONED`, and `PER_FRAME`;
- deterministic topology classes and frame-to-topology IDs;
- semantics-neutral frame groups;
- maximal ordered segments for trajectories;
- exact atomic-edge and decorated framework-edge transition differences;
- descriptive transient/confirmed segment labels without smoothing;
- schema-versioned serialization and catalog provenance.

Primary API

```
build_topology_catalog(
    collection: AtomisticFrameCollection,
    connectivity: AtomicConnectivityResult,
    mapping: FrameworkMapping,
    *,
    validation_rules: FrameworkValidationRules | None = None,
    projection_options: FrameworkProjectionOptions | None = None,
    catalog_options: TopologyCatalogOptions | None = None,
) -> TopologyCatalog
```

The principal public data structures are:

```
TopologyCatalogOptions
TopologyFrameGroup
TopologySegment
TopologyTransition
TopologyCatalog
```

Exact class identity

Topology classes are mapping-dependent exact structural equivalence classes. The authoritative key contains the framework-topology schema, mapping digest, PBC flags, ordered retained vertices and species, and sorted canonical `FrameworkEdgeKey` records. Digest equality narrows candidate comparisons but never replaces structured-key equality.

The whole-path orientation repair remains binding:

$$A - \mathrm{O} - \mathrm{S} - B \equiv B - \mathrm{S} - \mathrm{O} - A,$$

while

$$A - \mathrm{O} - \mathrm{S} - B \neq A - \mathrm{S} - \mathrm{O} - B.$$

Traversal direction, validation findings, projection diagnostics, and raw pre-gauge path provenance do not create separate topology classes.

Trajectories and ensembles

For a trajectory, the catalog stores both reusable topology classes and maximal contiguous segments. A sequence

```
A A A B B A A
```

contains two classes but three segments. Adjacent segment boundaries produce exact transitions. A persistence threshold labels short segments as transient; it never removes or merges them.

For an ensemble, frame groups are retained but no segments or transitions are created. Reordering an ensemble may change dense first-occurrence IDs, but it must not change the exact topology class set or counts.

Compression and provenance

Several atomic-connectivity states may map to one framework topology, such as changing spectator Na-O contacts with an unchanged LTA framework. The catalog retains every source state digest and the representative state used for each stored topology. Ring search therefore operates once per exact topology class, not once per frame or connectivity state.

Explicit non-responsibilities

The module does not:

- reconstruct atomic connectivity from coordinates;
- smooth topology histories;
- infer temporal order for ensembles;
- compare different atom orderings or cell bases approximately;
- enumerate rings;
- infer ring lineage, sites, cages, or dynamic regions.

`region_membership.py`

Responsibility

`region_membership.py` classifies where atoms are and whether originally assigned solid atoms remain attached to a tracked structural backbone. It is an independent analysis layer for heterogeneous systems and does not alter atom identity, connectivity scope, or topology definitions.

The module should support solid-liquid interfaces, solid-solid interfaces, multislab systems, dissolution, detachment, adsorption, and transfer between regions.

Primary inputs

- `AtomisticFrameCollection`;
- persistent reference atom groups or named original regions;
- optional geometric region definitions;
- optional anchor or seed atom indices;
- optional `AtomicConnectivityResult`;
- optional topology catalogs for material-specific backbone tracking;
- alignment and boundary-estimation options.

Primary outputs

A future region-membership result should preserve distinct per-frame properties, for example:

```
original_region_id
current_geometric_region_id
connected_to_reference_backbone
detached_from_reference_backbone
inside_interface_margin
```

It should also retain dynamic region boundaries, anchor transformations, detached fragment records, and complete provenance.

Analysis levels

The module should be designed in layers rather than attempting the most advanced classifier immediately.

Fixed geometric regions
Classify atoms using explicit Cartesian or fractional slabs, boxes, half-spaces, or user-defined masks. This mode assumes that drift has already been removed or that the selected coordinate system follows the material adequately.

Reference-aligned regions
Track anchor atoms and evaluate geometric regions in a translated or rigidly aligned reference frame. Translation-only alignment is preferable when a periodic slab should not physically rotate. A Kabsch-type rigid alignment may be useful for finite or freely rotating objects.

Topology-aware dynamic regions
Use connectivity or topology to identify the connected component containing specified seed atoms. This component defines the attached solid backbone. Robust surface positions may then be estimated from backbone atoms using quantiles, plane fits, or later local height fields. Original solid atoms outside the backbone are classified as detached rather than silently removed.

Important invariants

- original atom-region assignments are identity-based and persistent;
- current geometric region is a frame-dependent observable;
- structural attachment is defined through an explicit connectivity or topology model;
- a detached atom remains present in connectivity and topology diagnostics;
- global drift must not be confused with transfer across an interface;
- frame order is used only by explicitly temporal region models;
- region labels do not redefine framework roles or ring identities.

Explicit non-responsibilities

This module does not:

- define chemical bonds;
- alter `ConnectivityScope`;
- decide framework vertex or linker roles;
- repair or suppress topology transitions;
- infer transport connectivity merely from geometric proximity;
- replace ring-site or cage assignment.

It supplies complementary spatial and attachment labels that later site, transport, dissolution, and interface analyses may consume.

`primitive_ring.py`

Responsibility

`primitive_ring.py` receives one immutable `FrameworkTopology` and returns one immutable deterministic catalog of local periodic rings under an explicit ring family.

The implemented default is

`SHORTEST_PATH_PAIRS -> PRIMITIVE_NO_SHORTCUT`

and the retained secondary method is

`REMOVED_EDGE_SHORTEST -> EDGE_SHORTEST_SUBSET`

The removed-edge family is useful but is not a complete primitive catalog.

Primitive definition

For a lifted-simple zero-winding cycle C and $x, y \in C$, let the two cycle arcs have lengths ℓ_1 and ℓ_2. The cycle is primitive when

$$d_{\bar{G}}(x, y) = \min\{\ell_1, \ell_2\}$$

for every ring-vertex pair. Equivalently, no third path is strictly shorter than both cycle arcs. Under the primitive-ring definition of Goetzke and Klein and Yuan and Cormack [5, 6], this is equivalent to saying that C is not the symmetric-difference sum of two smaller cycles.

Only maximal half-cycle pairs are required. For

$$r = \left\lfloor \frac{k}{2} \right\rfloor,$$

even $k = 2r$ requires the r antipodal pairs, while odd $k = 2r + 1$ requires the k pairs separated by r ring edges.

Periodic covering graph

Let $\tilde{G}$ be the infinite lifted decorated multigraph and $G = \tilde{G}/\Lambda$ its finite quotient. A lifted vertex is

$$(u, \mathbf{n}), \qquad \mathbf{n} \in \mathbb{Z}^3.$$

For every quotient vertex u, the algorithm builds a bounded shortest-path index rooted at $(u, \mathbf{0})$. Targets retain their exact image shifts. Translation invariance gives

$$d_{\tilde{G}}((u, \mathbf{a}), (v, \mathbf{b})) = d_{\tilde{G}}((u, \mathbf{0}), (v, \mathbf{b} - \mathbf{a})).$$

One base-image index per quotient vertex orbit therefore answers every required bounded distance query.

Bounded periodic completeness theorem

Let $K \geq 2$ be the requested maximum ring size and

$$R = \left\lfloor \frac{K}{2} \right\rfloor.$$

Assume that:

- the lifted graph is locally finite, undirected, and unweighted;
- every quotient vertex is indexed from $(u, \mathbf{0})$ through depth R;
- all tied shortest paths are retained;
- every eligible path-pair combination is considered;
- exact lifted vertices and physical edge instances are preserved; and
- no resource limit truncates the search.

Then `SHORTEST_PATH_PAIRS` enumerates every translation orbit of lifted-simple, zero-winding primitive cycles of size at most K.

Translation-orbit representative

Take any finite lifted cycle C and a vertex $(u, \mathbf{n}) \in C$. Translation by $-\mathbf{n}$ maps C to an equivalent cycle containing $(u, \mathbf{0})$. Translation preserves length, lifted simplicity, graph distance, winding, and primitive status. Hence every ring orbit has a representative covered by one of the base-image source indexes.

Even cycles

For $|C| = 2r$, choose antipodal vertices u, v. The two ring arcs from u to v have length r. Primitiveness forbids a shorter path, so both arcs are tied shortest paths:

$$d_{\tilde{G}}(u, v) = r.$$

The even generator enumerates their internally disjoint pair and reconstructs C.

Odd cycles

For $|C| = 2r + 1$, choose root u and the opposite edge (v, w). Both ring paths from u to v and u to w have length r and are shortest. The odd generator enumerates the two paths and closes them with the exact lifted edge (v, w).

Finite reduction

Let

$$S = \{(u, \mathbf{0}) : u \in V(G)\}$$

and define the finite induced graph

$$H_K = [\tilde{G}x : d_{\tilde{G}}(x, S) \leq K\}].$$

Local finiteness and finite quotient size make H_K finite.

Every translated cycle C of size $k \leq K$ containing a vertex in S lies within radius $\lfloor k/2 \rfloor$ of S. If P is a strict shortcut between cycle vertices, then

$$|P| < \left\lfloor \frac{k}{2} \right\rfloor,$$

and every vertex of P lies at distance less than $k \leq K$ from S. Therefore H_K contains both the cycle and every shortcut witness that can change its primitive classification.

Any finite decomposition of C into two smaller cycles lies in some finite subgraph of $\tilde{G}$. The finite-graph primitive-ring equivalence then produces a strict shortcut; the preceding radius bound places that witness in H_K. Conversely, any shortcut in H_K is also a shortcut in $\tilde{G}$. Thus primitive classification in H_K and in the infinite lift agree for cycles through size K.

Conclusion
After canonicalization under cyclic rotation, reversal, and lattice translation, the catalog contains exactly one identity for each generated ring orbit. An untruncated result may therefore state:

> Complete for all lifted-simple, zero-winding primitive-ring translation orbits in the requested size interval under the declared unweighted decorated framework model.

This bounded periodic proof is an original `mdstats` derivation. The primitive ring definition and finite/infinite network context are attributed to Goetzke and Klein and to Yuan and Cormack [5, 6].

Candidate generation and classification

For even size $k = 2r$, enumerate pairs of internally lifted-vertex-disjoint tied shortest paths of length r between exact antipodes. For odd size $k = 2r + 1$, enumerate two shortest root paths of length r to the endpoints of one exact closing edge. Two-member parallel-edge rings and triangles are explicit parity base cases.

Every candidate must satisfy exact lifted continuity, lifted vertex simplicity, physical edge-instance uniqueness, zero winding, requested size, and complete decorated-path orientation. The classifier verifies the remaining maximal half-cycle pairs against the shared index. Optional external-shortcut witnesses are diagnostic only.

Resource and provenance semantics

The search records limits on lifted states, tied paths, path-pair combinations, candidates, and accepted rings. Limits are transactional: an incomplete source or anchor is not silently represented as complete.

The result records search method, ring family, requested bound, achieved completeness, truncation diagnostics, topology provenance, canonical schema, and stable digest.

Algorithmic attribution

Horton and Vismara [3, 4] motivate shortest-path cycle prototypes. Goetzke and Klein and Yuan and Cormack [5, 6] supply primitive-ring definitions and the finite/infinite topological-network context. The following are `mdstats`-specific:

- lazy lifted decorated-multigraph traversal;
- the translation-orbit completeness proof;
- the finite-radius reduction H_K;
- parity-specific bounded candidate assembly;
- exact physical edge-instance identity;
- transactional resource semantics; and
- deterministic canonical serialization.

Implementation status

Stages S4R.0-S4R.5 are complete: terminology and schema v2, bounded lifted shortest-path indexes, parity generators, primitive classification, deterministic catalog construction, compatibility, and validation all pass. The current Na-LTA catalog through size eight is

$$36 \times 4\mathrm{R} + 40 \times 6\mathrm{R} + 6 \times 8\mathrm{R}.$$

No separate LTA-specific 8-ring detector is permitted downstream.

Explicit non-responsibilities

The module does not:

- compute ring geometry;
- classify strong rings;
- infer faces, tiles, windows, sites, or cages;
- include nonzero-winding quotient loops as local rings; or
- promote an `EDGE_SHORTEST_SUBSET` catalog to primitive completeness.

`_periodic_graph.py`

This private helper is **not** the first implementation stage. It is extracted after the symmetry and strength prototypes identify repeated operations.

Its maximum responsibility is deliberately small:

- exact integer lattice-shift arithmetic;
- translation and relative-image conversion for existing lifted-reference records;
- canonical reversal of undecorated periodic endpoint pairs; and
- physical lifted framework-edge instance anchors.

`LiftedAtomRef` and `LiftedVertexRef` retain separate semantic types even though they share fields. The former may refer to linker atoms; the latter is restricted to projected framework vertices. Generic helpers may operate through a protocol or accessor but must not collapse the public types into one alias.

Decorated framework-edge reversal remains in `framework_topology.py`. Ring canonicalization remains in `primitive_ring.py`. Boundary parametrization, embedded-face orientation, and cell attaching maps remain in their owning modules. No circular import is permitted, and the refactor must preserve existing public imports, serialization, deterministic ordering, and digests.

`periodic_net_view.py`

`periodic_net_view.py` defines the exact periodic multigraph whose symmetry and natural tiling are being computed.

Its primary immutable result is conceptually

`PeriodicNetView`

bound to one `FrameworkTopology` and one explicit `NetViewPolicy`.

For the **first backend**, the view is a signature projection only:

$$V_{\mathrm{view}} = V_{\mathrm{framework}}, \qquad E_{\mathrm{view}} = E_{\mathrm{framework}}.$$

It may change only the deterministic vertex and edge signatures that automorphisms must preserve. It must not remove vertices, remove edges, contract linker paths, merge parallel edges, or otherwise change the graph on which the existing `PrimitiveRingCatalog` was computed. If a future scientific workflow requires a structurally filtered or contracted net, that altered graph is a new topology source and requires its own compatible primitive-ring catalog.

The policy therefore provides deterministic signatures for:

- each projected framework vertex orbit; and
- each projected framework edge orbit.

Ignoring `rule_id`, Si/Al identity, linker species, or another decoration means that objects differing only in that ignored signature may be permuted by an automorphism. Their graph records remain distinct.

The view stores exact mappings between view-local vertex/edge orbits and source `FrameworkTopology` records. Its digest identifies the exact symmetry policy and source graph used by properness and natural-tiling results. Primitive-ring identity, however, remains rooted in the source topology graph digest plus `PrimitiveRingKey`; the net-view digest is additional symmetry/tiling provenance, not a replacement for the graph on which the ring key was defined.

For conventional LTA natural tiling, the default policy is the unlabeled framework T-net:

- Si and Al receive one common topological vertex signature;
- oxygen linkers remain represented by the same projected framework edge orbits;
- Na, Li, K, and other spectators are excluded upstream; and
- parallel framework edges remain distinct even when decorative signatures are ignored.

A chemically decorated policy may instead preserve Si/Al and selected edge attributes.

Before symmetry or tiling, the view computes basic periodic-net preconditions:

- quotient-graph connected components;
- translation subgroup generated by closed-walk cycle gains;
- exact translation rank; and
- finite subgroup index when the component has full periodic rank.

The finite index is essential because a quotient-connected rank-three graph can still lift to multiple disconnected translation cosets. The first natural-tiling backend therefore requires

$$N_{\text{quotient components}} = 1, \qquad \operatorname{rank} \Lambda_G = 3, \qquad [\mathbb{Z}^3 : \Lambda_G] = 1.$$

Molecular, one-periodic, two-periodic, quotient-disconnected, and index-greater- than-one structures remain valid diagnostic views but require separate algorithms and are rejected explicitly by the first three-periodic tiling backend.

periodic_barycentric.py

`periodic_barycentric.py` owns the reusable exact rational equilibrium placement of one `PeriodicNetView`. The result is

`PeriodicBarycentricPlacement`

and stores the view/topology digests, deterministic anchor atom, exact $\mathbb{Q}^3$ coordinates, collision pairs modulo $\mathbb{Z}^3$, rational coefficient-growth diagnostics, and a canonical digest.

For quotient vertex i,

$$x_i = \frac{1}{\deg(i)} \sum_{(i,j,\mathbf{t})} (x_j + \mathbf{t}),$$

with one anchor fixed to remove translational gauge. Dense exact elimination is protected by explicit vertex-count and fraction-bit resources. The object is shared by symmetry discovery and the implemented embedding layer; no consumer may copy or privately re-solve the same equilibrium system.

This result is not yet a Euclidean embedding. It provides fractional/topological coordinates only. `PeriodicNetEmbedding` must additionally choose and verify a positive-definite lattice metric.

periodic_net_embedding.py

`periodic_net_embedding.py` owns the authoritative first-backend Euclidean realization of an exact `PeriodicNetView`. Construction accepts a complete `PeriodicNetSymmetryDiscovery`, not an arbitrary symmetry subgroup, so the metric and coordinates are checked against the complete automorphism group in the supported domain.

The persistent result is

`PeriodicNetEmbedding`

and stores:

- exact view, topology, symmetry, barycentric-placement, and ring-independent discovery-certificate digests;
- the deterministic anchor and exact rational fractional coordinates;
- the exact primitive integral lattice Gram matrix;
- the projected-edge model and stable source edge keys;
- exact minimum and maximum projected-edge squared lengths; and
- one canonical embedding digest.

For each projected quotient edge

$$e = (i, j, \boldsymbol{\delta}_e), \qquad \mathbf{d}_e = \mathbf{x}_j + \boldsymbol{\delta}_e - \mathbf{x}_i,$$

the first backend constructs

$$C = \sum_e \mathbf{d}_e \mathbf{d}_e^T, \qquad G_{\mathbb{Q}} = C^{-1}.$$

Denominators are cleared and the common integer divisor is removed to produce the primitive integral Gram matrix G. Complete edge permutation/orientation data imply

$$A_g C A_g^T = C \quad \Rightarrow \quad A_g^T G A_g = G,$$

and the implementation verifies this identity exactly for every operation. Under a unimodular basis change P, the metric transforms as

$$G' = P^T G P,$$

so the Euclidean shape is not an artifact of the chosen periodic indexing basis.

The numerical Cartesian cell uses row lattice vectors and the lower-triangular Cholesky factor of

$$\bar{G} = G/(\det G)^{1/3},$$

which fixes unit cell volume. Cartesian values are derived numerical geometry; the rational coordinates and exact Gram matrix remain authoritative.

The v1 edge model is

`ProjectedEdgeCurveModel.STRAIGHT_SEGMENT`

and `edge_segment()` returns a transient source-bound `EmbeddedStraightEdgeSegment`. Distinct multiedges are never merged. If two distinct quotient edges become the same straight segment modulo translation and reversal, construction fails and a future distinct-curve backend is required.

Stage 8A rejects collided vertices, zero-length projected edges, singular edge covariance, failed exact symmetry equivariance, and coincident distinct straight edges. It does not yet certify arbitrary periodic edge-edge nonintersection; that requires Stage 8B's image-labelled extended-object broad phase and exact segment predicates.

A relaxed or finite-temperature MD frame does not redefine the natural tiling. Such frames are mapped onto the accepted topological objects only for downstream ring, tile, aperture, site, and cage geometry. Primitive one- or two-member rings remain valid graph objects but cannot become nondegenerate straight-segment face boundaries unless a later curve model supplies distinct embedded arcs.

`periodic_cycle.py`

`periodic_cycle.py` supplies lightweight, source-bound operations on canonical primitive rings. It creates no new scientific catalog and no public chain algebra.

The design separates physical placement from boundary parametrization.

A physical translated occurrence is

```
RingPlacement(
    topology_graph_digest,
    ring_key,
```

```
    image_shift=(0, 0, 0),
)
```

where `ring_key` is the stable `PrimitiveRingKey`, not a catalog-local `ring_id`. Within a bound index, the key may resolve to a dense integer for speed.

The translation is defined relative to the deterministic canonical lifted representative associated with the key. Let $\hat{R}(q)$ be the canonical step sequence selected by the primitive-ring canonicalization, reconstructed as a lifted walk and translated so that its first lifted vertex has image shift $\mathbf{0}$. Then

$$R(q, \mathbf{t}) = \hat{R}(q) + \mathbf{t}.$$

This canonical anchor is part of the ring-placement contract. Increasing the search bound may change dense IDs but must not change $\hat{R}(q)$ for any pre-existing key q.

A boundary parametrization is

```
CycleParameterization(
    start_vertex_index=0,
    orientation=+1,
)
```

For an n-ring, the parametrized vertex and edge positions are defined exactly by

$$p_V(k) = c + \epsilon k \pmod{n},$$

and

$$p_E(k) = \begin{cases} c + k \pmod{n}, & \epsilon = +1, \\ c - k - 1 \pmod{n}, & \epsilon = -1. \end{cases}$$

This convention removes the ambiguity in the phrase "rotate and traverse backward."

An oriented boundary view combines the two:

```
OrientedRingView(
    placement,
    parameterization,
)
```

The view lazily yields lifted vertices, source ring-step indices, traversal orientations, and physical edge instances. The $2n$ parametrizations of one n-ring are not distinct physical placements.

Persistent or cross-stage ring identity is

```
(topology graph digest, PrimitiveRingKey)
```

The `PeriodicNetView` digest is attached as additional symmetry/tiling provenance. It does not replace the topology graph digest because the key is defined from the source framework-edge records.

A local integer `ring_id` is only an index into one exact catalog and may change when the ring-size bound is enlarged.

For strong-ring analysis, Stage 5 provides exact private modulo-two support over physical `LiftedEdgeInstanceRef` objects. Distinct translated instances of one quotient-edge orbit never cancel unless they are the same physical edge. The finite solver may encode the participating support as a temporary local bitset.

A public `PeriodicEdgeChain` is deferred until `periodic_cell_complex.py`, where the source graph, canonical edge orientation, coefficient ring, and boundary operators are fully specified.

The exact action of a net automorphism on a ring is represented by an ordered occurrence map, not merely a token-level match. Conceptually it records:

```
RingOccurrenceMap(
    topology_graph_digest,
    source_placement,
```

```
    target_placement,
    source_vertex_position_to_target_position,
    source_step_position_to_target_position,
    parameterization=CycleParameterization(...),
)
```

The explicit occurrence permutations resolve repeated edge tokens and symmetric self-alignments. Derived offset/orientation values are conveniences; the ordered maps are authoritative for group composition.

`primitive_ring_index.py`

`primitive_ring_index.py` provides a lightweight computational index over one exact `PrimitiveRingCatalog`.

Its core object is

`PrimitiveRingIndex`

which is immutable, cacheable, and initially nonserialized. It is bound to the ring-catalog digest and canonical schema. It does not repeat scientific provenance or receive an independent persistent identity.

The core index does **not** require `FrameworkTopology`. The ring catalog already contains the canonical projected edge keys required for:

- edge-orbit to ring-step inverse incidence;
- framework-vertex to ring-occurrence inverse incidence;
- physical edge-instance anchors; and
- translation of ring placements onto a requested edge instance.

Consumers requiring expanded linker paths or Cartesian geometry use a separate validated context, conceptually

```
FrameworkRingContext(topology, rings, index)
```

which checks source digests once.

The implemented P1 index stores stable-key-to-dense-ID resolution and exact edge-orbit-to-ring-step occurrence incidence. A framework-vertex inverse index may be added later only when a concrete consumer requires it. The index does not store copied boundaries, translated ring occurrences, global pair contacts, persistent quotient-edge vectors, or all vertex-pair combinations.

The supported Stage-5 records are:

```
LiftedEdgeInstanceRef
RingPlacement
CycleParameterization
RingEdgePlacement
PrimitiveRingIndex
```

Canonical ring-step occurrence buckets are private index implementation detail. They are intentionally not exported as a public record type.

The implemented constructors/queries are:

```
build_primitive_ring_index(...)
ring_placements_covering_edge(...)
```

For quotient edge $e = (i, j, \Delta)$, physical instance anchor $\mathbf{a}$ denotes the edge from $(i, \mathbf{a})$ to $(j, \mathbf{a} + \Delta)$. A reverse-traversed canonical ring step therefore has anchor $\mathbf{s} - \Delta$, not merely its source lifted image $\mathbf{s}$. Aligning a canonical occurrence anchored at $\mathbf{a}_k$ to a requested instance anchored at $\mathbf{a}$ gives the unique ring translation

$$\mathbf{t} = \mathbf{a} - \mathbf{a}_k.$$

The central placement query is:

> Which translated placements of allowed primitive-ring keys use this exact physical framework-edge instance?

It aligns each canonical occurrence of the same edge orbit to the requested anchor and returns unique `RingPlacement` results plus the aligned source-step index and traversal orientation. Vertex-placement and shared-boundary-path queries are generated lazily for selected rings.

Naked `ring_id` and `edge_index` values must not escape a source-bound operation without either the owning context or a stable key/digest. A mismatched catalog is an error, not a best-effort interpretation.

The Stage-5 identity/view API remains provisional only until the three concrete consumer implementations are compared for actual duplicated operations:

1. exact translated ring placement over physical edge instances (P1);
2. exact automorphism-induced ordered ring-occurrence mapping (P2); and
3. exact finite GF(2) cancellation over translated physical edge support (P3).

All three prototypes now pass. Only operations demonstrably shared by these implementations are extracted into private helpers before the lightweight Stage-5 API is frozen.

`primitive_ring_cancellation.py`

`primitive_ring_cancellation.py` implements the Stage 5-P3 exact finite cancellation prototype over translated primitive-ring support.

The authoritative support basis element is the complete source-bound physical edge instance

```
LiftedEdgeInstanceRef(topology_graph_digest, edge_key, anchor_shift)
```

rather than a quotient-edge index alone. Dense edge indices remain transient acceleration handles inside `PrimitiveRingIndex`. For a canonical ring step anchored at $\mathbf{a}_k$ and placement translation $\mathbf{t}$, the translated support entry is

$$(e_k, \mathbf{a}_k + \mathbf{t}).$$

Thus two translated instances of the same quotient-edge orbit remain different basis elements and cannot cancel spuriously.

The implemented transient derived record

```
RingPlacementSupport
```

stores the sorted duplicate-free exact physical edge support of one `RingPlacement`. It has no independent persistent identity and is always reconstructible from the source-bound `PrimitiveRingIndex`.

For one explicit finite target/candidate problem, the solver builds the finite physical-edge universe, encodes every support as a temporary Python-integer bit vector, and performs deterministic Gaussian elimination over $\mathrm{GF}(2)$. The strong-ring sum/symmetric-difference concept follows Goetzke and Klein and Yuan and Cormack [5, 6]; the temporary bitset/elimination representation is an mdstats implementation choice using standard finite-field linear algebra.

The public prototype query is

```
solve_finite_ring_cancellation(index, target_placement, candidate_placements)
```

where every supplied candidate must be strictly smaller than the target. The two exact statuses are

```
DECOMPOSITION_FOUND
NOT_IN_SUPPLIED_SPAN
```

A positive result carries one deterministic `RingCancellationWitness` whose component placements re-verify to empty exact physical-edge symmetric difference. A negative result is complete only for the supplied finite candidate span. It is **not** `STRONG` or `STRONG_IN_DOMAIN`.

This distinction is normative. Later `ring_strength.py` must separately prove that a declared immutable candidate domain was enumerated completely, that the source primitive-ring catalog is complete for every admitted component size, and that no resource truncation occurred before promoting finite algebraic failure to a strength certification.

The P3 gate deliberately does not enumerate candidate placements, impose component-count or placement-radius bounds, serialize support chains, or expose a public `PeriodicEdgeChain`. Those belong to later strength and cell-complex layers.

net_symmetry.py

`net_symmetry.py` owns only the finite group of normalized periodic-net automorphisms modulo common lattice translations. Every operation belongs to one exact `PeriodicNetView` and acts as

$$g(i, \mathbf{n}) = \big(\pi_g(i), A_g\mathbf{n} + \boldsymbol{\tau}_i\big), \qquad A_g \in GL(3, \mathbb{Z}).$$

The deterministic anchor gauge imposes

$$\boldsymbol{\tau}_{i_0} = \mathbf{0}.$$

Exact composition is

$$A_{g\circ h} = A_g A_h, \qquad \tau_i^{g\circ h} = A_g \tau_i^h + \tau^g_{\pi_h(i)}.$$

Because each product is renormalized, the persistent group stores the exact common-translation cocycle

$$\hat{g}\hat{h} = T_{\mathbf{c}(g,h)}\widehat{gh}.$$

The persistent `PeriodicNetSymmetry` contains normalized operations, multiplication and inverse tables, the cocycle table, identity, vertex/edge orbits, source digests, and a canonical result digest. It contains no primitive- ring data, barycentric coordinates, discovery workspace, embedding, faces, or tiles.

Finite closure uses validated generators and their inverses in a Cayley-style breadth-first enumeration. Exceeding `max_operations` publishes no partial group. Exact periodic-net representation and combinatorial symmetry follow Chung, Hahn, and Klee [1] and Delgado-Friedrichs and O'Keeffe [7].

primitive_ring_symmetry.py

`primitive_ring_symmetry.py` is a derived index that applies one `PeriodicNetSymmetry` to one exact `PrimitiveRingCatalog`. It is bound to:

- `PeriodicNetSymmetry.digest`;
- `PeriodicNetView.digest`;
- topology graph digest;
- `PrimitiveRingCatalog.digest`; and
- the ring catalog completeness/truncation metadata.

`PrimitiveRingSymmetryIndex` stores each stable ring key once. Every action cell stores only a target ring position, target image shift, and exact `CycleParameterization`. Ring orbits and stabilizers are position-based internally and may be exposed as stable keys through accessors.

For every operation pair and represented ring, construction verifies the cocycle-corrected induced-action homomorphism. Missing transformed rings, incompatible catalog provenance, or exceeded composition-check resources fail transactionally.

This split prevents the core net group from scaling with ring-bound refinement and prevents ring actions from being reused against a different primitive-ring catalog merely because the topology digest matches.

ring_strength.py

`ring_strength.py` classifies primitive rings under finite, explicitly declared sums of strictly smaller primitive-ring placements. For target support $[C]$,

$$[C] \in \mathrm{span}_{\mathrm{GF}(2)}\{[R_1], \ldots, [R_N]\}$$

is a weak-ring certificate in the declared finite domain.

The mathematical domain and execution resources remain separate:

```
RingStrengthDomain(
    target_ring_key,
    max_component_size,
```

```
    placement_domain,
)
RingStrengthResources(
    max_candidate_placements,
    max_search_nodes,
    max_support_terms,
    max_matrix_bits,
    max_provenance_bits,
)
```

No independent `max_component_count` belongs to the mathematical domain. Over a finite candidate set, each coefficient is zero or one and existence is ordinary $\mathrm{GF}(2)$ span membership.

The module now enforces a strict persistence boundary:

- `RingStrengthSearchWorkspace` transiently owns the complete deterministic candidate-placement tuple;
- `RingStrengthResult` persistently owns source digests, target, domain, resources, status, diagnostics, candidate-set digest, and optional compact witness;
- `RingStrengthWitness` contains only the exact positive decomposition; and
- `RingStrengthCatalog` contains persistent results, not workspaces.

The exact statuses remain

```
WEAK_CERTIFIED
STRONG_IN_DOMAIN
UNRESOLVED_TRUNCATED
UNRESOLVED_SOURCE_INCOMPLETE
```

A weak witness is verified by exact physical lifted-edge parity. A negative result means only that the fully exhausted finite domain contains no solution. Resource or source incompleteness never becomes a negative theorem.

Persistent deserialization is verification, not merely hash checking. By default, `RingStrengthResult.verify(index)` re-enumerates the deterministic finite domain, checks the candidate-set digest, reruns exact cancellation, and compares the canonical scientific result. A modified payload with a recomputed JSON digest is therefore rejected if its witness or bounded theorem is false.

Before exact elimination, the backend estimates support-matrix and provenance storage:

$$B_{\mathrm{matrix}} = NE, \qquad B_{\mathrm{provenance}} = N\min(N, E),$$

where N is candidate count and E represented physical edge count. Exceeding either declared bound returns `UNRESOLVED_TRUNCATED` before large Python-integer allocation.

The primitive source catalog must be lower closed through the active component size and untruncated. Unresolved strength propagates to face and tiling certification.

`_periodic_spatial.py`

`_periodic_spatial.py` is a private Euclidean broad-phase backend for **bounded extended objects** such as ring boundaries, spanning surfaces, triangulated face witnesses, tile volumes/shells, and cage/portal surfaces. It is deliberately separate from the existing atomic cell-list implementation because an extended object may occupy many bins, cross periodic boundaries, and require a consumer-specific exact predicate after candidate generation.

The backend answers only:

> Which periodic image placements of bounded embedded objects cannot yet be ruled out by conservative Euclidean support bounds?

It does **not** decide graph reachability, primitive-ring adjacency, topological connectivity, linking, face validity, or tile overlap. Those meanings belong to the consumer.

Every input object is represented in one **continuous lifted support**. Its vertices and geometric primitives may lie outside the reference cell, but they form one connected/unwrapped realization rather than a wrapped coordinate cloud. This prevents a boundary-crossing ring, surface, or tile from acquiring an artificially large or disconnected support merely because coordinates were wrapped.

For bounded objects A and B and cell matrix H, periodic images are

$$B_{\mathbf{n}} = B + H\mathbf{n}, \qquad \mathbf{n} \in \mathbb{Z}^3.$$

The consumer supplies conservative support bounds and a finite broad-phase **admission rule**. Distance queries are one important special case:

$$\mathcal{S}_r(A, B) = \{\mathbf{n} : \operatorname{dist}(A, B + H\mathbf{n}) \le r\}.$$

The generic backend is not restricted to one scalar object-distance cutoff. Examples include:

- distance/proximity: inflated supports may lie within a declared cutoff;
- surface intersection or penetration: conservative support volumes overlap;
- linking certificates: a ring support and a chosen spanning-surface support may intersect;
- tile overlap: conservative **filled-volume** supports overlap or one may contain the other; and
- cage/portal queries: consumer-specific conservative proximity or containment bounds.

For every query, the builder returns a finite translation stencil $\hat{\mathcal{S}}$ guaranteed to contain every image that can satisfy the exact consumer predicate under the declared support rule. Fixed shells such as $[-1, 1]^3$ are forbidden as correctness assumptions. For skewed cells, integer translation ranges are derived from reciprocal/fractional geometry and filtered by tighter Cartesian support tests.

For small cells or few objects, the backend may use direct object-pair enumeration plus the complete translation stencil. For larger cells it builds a dedicated extended-object linked-cell subdivision. The linked-cell idea is adapted from the classical neighbor-search decomposition of Quentrec and Brot [17]; the extension to continuous lifted extended objects, conservative multi-bin occupancy, explicit periodic image labels, automatic grid selection, and query-specific support rules is an `mdstats` design.

Grid selection uses:

- perpendicular cell heights / reciprocal geometry;
- conservative object support extents or bounding radii;
- the largest declared broad-phase interaction/support margin;
- expected multi-bin insertion cost;
- expected candidate-bin work; and
- memory limits.

Correctness is independent of the selected grid. The deterministic optimizer builds a bounded set of candidate integer grids, computes the exact metric cell-neighbor stencil for each, estimates insertions and candidate work, rejects memory-unsafe layouts, and chooses the minimum-cost candidate with a deterministic tie-break. It may therefore choose one bin per axis for a small problem and finer subcells for a large sparse cell.

Extended objects are inserted into every bin touched by their conservative lifted support unless a proved specialization is used. Because a support can cross the unit cell or span multiple cells, occupancy records retain both object identity and the occupancy image shift. Candidate identity is canonicalized as

```
(object_i, object_j, relative_image_shift)
```

with

$$(i, j, \mathbf{n}) \sim (j, i, -\mathbf{n}).$$

Self-image candidates $(i, i, \mathbf{n})$ with $\mathbf{n} \neq \mathbf{0}$ are valid and must not be collapsed into the zero-image self pair. Deterministic sign canonicalization may retain one of $\mathbf{n}$ and $-\mathbf{n}$ for undirected self-image predicates.

A typical exact-query pipeline is

```
continuous lifted object supports
-> complete periodic translation stencil
-> optional extended-object linked-cell broad phase
-> conservative object-level rejection
-> primitive-level spatial index (segments / triangles / tetrahedra)
-> robust exact consumer predicate
```

The same candidate generator may therefore accelerate ring–surface intersection, face-witness intersection, framework penetration, tile-volume overlap, self-image checks, containment, and finite-range proximity without conflating their scientific interpretations.

Reuse of the existing Verlet validity theorem

The extended-object **cell list is separate**, but the mathematical cache-validity kernel is shared with the implemented atomic Verlet backend.

Assume an embedded object has fixed graph/mesh connectivity and each point of each edge/triangle/simplex is obtained by affine interpolation of its tracked vertices. If every tracked vertex of object A moves by at most

$$\delta_A = \max_{i \in V_A} \|\Delta \mathbf{x}_i\|,$$

then every point of the object moves by at most δ_A. Therefore the Hausdorff displacement is at most δ_A, and for two explicit image placements,

$$|d_t(A, B_{\mathbf{n}}) - d_0(A, B_{\mathbf{n}})| \le \delta_A + \delta_B \le 2\delta_{\max}.$$

For a distance-buffered candidate rule built with

$$r_{\text{list}} = r_{\text{cut}} + r_{\text{skin}},$$

a sufficient fixed-cell reuse condition is

$$2\delta_{\max} < r_{\text{skin}},$$

or the sharper pairwise condition

$$\delta_A + \delta_B < r_{\text{skin}}.$$

Translation, rigid rotation, bending, and internal PL deformation are all covered by the same vertex-displacement bound. The classical neighbor-list buffering idea comes from Verlet [15,16]; the fixed-connectivity extended-object displacement proof and its use for graph/mesh supports are `mdstats` derivations.

For variable cells, the implementation reuses the existing deformation-aware S3 margin in generalized endpoint-budget form:

$$M_{AB}(t) = \sigma_{\min}(F_t)(r_{AB} + r_{\text{skin}}) - r_{AB} - u_A^{\max}(t) - u_B^{\max}(t).$$

The same singular-value contraction bound, cell-conditioning checks, safety tolerance, and rebuild semantics used by the implemented atomic cache are reused.

Reuse is at the level of the **validity theorem/kernel**, not the atomic `VerletPairCache`'s unique-image/MIC assumptions. Extended-object caches may contain several explicit periodic images of the same object pair. Candidate identities therefore retain relative image shifts, and omitted-image safety is proved per explicit image/support rule.

For non-distance predicates such as intersection or overlap, cache reuse is allowed only when the consumer supplies a buffered conservative support rule whose motion under vertex/cell deformation is bounded by the same displacement kernel. The validity kernel decides whether the conservative candidate superset remains complete; it never asserts the exact predicate itself.

The first extended-object implementation keeps a separate cache payload containing object IDs, image shifts, support bounds, and consumer-query metadata. A later refactor may merge more of the atomic and extended-object cache infrastructure only if the two concrete implementations demonstrate identical invariants.

Any change in object topology, mesh connectivity, witness identity, tracked vertex set, support rule, interaction request, embedding, or periodic-cell compatibility invalidates the cache regardless of displacement.

`periodic_edge_intersection.py`

`periodic_edge_intersection.py` owns the first exact global geometric certificate for `ProjectedEdgeCurveModel.STRAIGHT_SEGMENT`. It is distinct from the private `_periodic_spatial.py` candidate workspace and from the persistent `PeriodicNetEmbedding`.

For two explicit segment placements

$$P(t) = \mathbf{p} + t\mathbf{r}, \qquad Q(u) = \mathbf{q} + u\mathbf{s}, \qquad 0 \le t, u \le 1,$$

the implementation uses exact rational cross products and dot products. Nonparallel lines are tested by exact coplanarity and exact parameters; parallel lines are projected onto a nonzero component to distinguish no contact, one-point contact, and positive-length collinear overlap. Because the Cartesian cell map is invertible, this rational fractional-coordinate predicate is equivalent to the Cartesian predicate.

Endpoint contacts are interpreted through exact lifted graph identity. Only a contact between the same `LiftedVertexRef` is allowed. The public `PeriodicEdgeIntersectionCertificate` records source digests, broad-phase diagnostics, allowed-contact counts, and every forbidden exact contact. Its schema is replay-verified against the exact view and embedding during deserialization.

`face_candidates.py`

`face_candidates.py` converts eligible primitive-ring placements into possible scientific face placements on one exact `PeriodicNetEmbedding`. No candidate is called essential at this stage.

Natural-tiling topology is constructed only on the validated, provenance-bearing reference embedding. A relaxed or thermally distorted trajectory frame does not define the tiling unless it independently satisfies the same embedding contract; ordinary compatible trajectory frames are used later only for descriptive ring and tile geometry.

The architecture separates the **scientific face** from the auxiliary geometric witness used to certify it.

```
FacePlacement(
    ring_placement,
    orientation,
)

FaceEmbeddingWitness(
    face_placement,
    witness_id,
    triangulation_or_surface_data,
)
```

A `FacePlacement` is the candidate topological 2-cell. A `FaceEmbeddingWitness` is one embedded piecewise-linear disk or other admitted surface realization proving that the boundary can be embedded compatibly. Multiple triangulations or disk witnesses may certify the same scientific face and must not create duplicate natural tilings merely because their auxiliary meshes differ. If distinct witnesses genuinely induce different scientific cell complexes, those complexes remain distinct at the cell-complex level.

The module reconstructs continuous lifted ring boundaries, rejects boundary self-intersection, searches a declared bounded family of embedded-disk witnesses, detects framework penetration and periodic self-intersection, and evaluates linking and witness compatibility. Robust orientation and sidedness decisions use adaptive or exact-sign predicates following Shewchuk [12]. Triangle-intersection kernels may follow Moller [13], with degeneracies delegated to robust predicates. The bounded spanning-disk policy is motivated by Hass, Snoeyink, and Thurston [14].

Linking and catenation semantics

Two distinct questions are kept separate.

For an oriented ring C_1 and an oriented spanning surface S_2 with $\partial S_2 = C_2$, the algebraic intersection number

$$I(C_1, S_2) = \mathrm{lk}(C_1, C_2)$$

is independent of the chosen Seifert surface under the usual disjoint-boundary hypotheses. A nonzero value is therefore a rigorous certificate that the two ring components are linked. Intersection-theoretic formulations of linking and higher-order linking are standard; an algorithmic simplicial treatment is given by Hsieh, Kauffman, and Tsau [18].

However, the geometric intersection of two **particular** spanning-disk witnesses D_1 and D_2 has a different meaning:

- $D_1 \cap D_2 \neq \varnothing$ means those particular witness choices are incompatible; it does not by itself prove intrinsic catenation of the boundary rings;
- disjoint embedded disk witnesses provide an explicit unlinking witness for the two-component disk-bounding case; and
- $\mathrm{lk} = 0$ does not by itself certify unlinking, because algebraic intersection can cancel in nontrivial links.

The first backend therefore distinguishes at least:

```
PROVEN_LINKED_NONZERO_INTERSECTION
WITNESS_PAIR_INCOMPATIBLE
DISJOINT_DISK_WITNESS
UNRESOLVED_LINKING
```

No bounded failure to find disjoint disks is promoted to a topological linking theorem.

All expensive Euclidean tests use `_periodic_spatial.py`. Linking-number candidates use ring–surface support overlap followed by oriented segment–triangle (or corresponding robust primitive) intersections. Witness-pair compatibility uses surface–surface intersection. These can be framed geometrically as zero-distance or intersection predicates, but the scientific interpretation of the result remains consumer-specific.

Compatibility is represented as a finite constraint system over scientific face placements and their admissible witnesses, not only a simple pairwise graph. The result may contain:

- unary invalidity certificates;
- pairwise forbidden witness assignments;
- higher-order forbidden tuples;
- symmetry-linked scientific face assignments;
- witness-equivariance relations when required; and
- unresolved constraints when bounded geometry is insufficient.

Auxiliary triangulations are certificates, not persistent face identity. Symmetry must preserve the scientific face set and may map one valid witness to a different equivalent witness. It need not reproduce identical triangulation combinatorics.

Failure to find a disk in the declared finite witness family yields `UNRESOLVED`, not knottedness or catenation. All unresolved strength, embedding, linking, or compatibility assumptions propagate to tiling certification.

Periodic-image enumeration uses complete query-specific support stencils. Large cells may use automatic linked subcells, and repeated geometric queries may reuse a candidate cache only under a proved buffered support rule and the shared deformation-aware Verlet validity kernel. A fixed $3 \times 3 \times 3$ image shell is never a completeness proof.

`periodic_cell_complex.py`

`periodic_cell_complex.py` is the first layer that owns formal source-bound cellular chain algebra. It consumes one compatibility-safe selected witness per scientific face and caller-supplied oriented tile shells. It does not infer shells from local face sectors.

The finite quotient stores translated cell terms $(c, \mathbf{n})$ with $\mathbf{n} \in \mathbb{Z}^3$ and integer coefficients. The operators

$$C_3 \xrightarrow{\partial_3} C_2 \xrightarrow{\partial_2} C_1 \xrightarrow{\partial_1} C_0$$

retain exact attaching translations and verify

$$\partial_2 \partial_3 = 0, \qquad \partial_1 \partial_2 = 0.$$

Self-image edges, faces, and tile adjacencies are first-class incidences rather than special cases. The quotient must satisfy

$$N_0 - N_1 + N_2 - N_3 = \chi(T^3) = 0.$$

For each proposed tile orbit, Stage 9 expands translated face boundaries into physical edge and vertex occurrences. Every edge occurrence must have exactly two oppositely signed face incidences, the face-adjacency graph must be connected, and the lifted shell must satisfy

$$\chi(\partial T) = V - E + F = 2.$$

Thus the accepted finite shell is connected, orientable, nonbranching, and genus zero under the declared cellular assumptions. These are validation conditions, not a shell-discovery algorithm.

The scientific `PeriodicCellComplex` digest includes face placements, tile shells, and boundary operators. Selected witness digests are retained only as construction provenance and are excluded from scientific identity.

The separate `PeriodicPartitionCertificate` takes an explicit periodic tetrahedral mesh whose elements are assigned to translated scientific tile placements. Auxiliary vertices are exact rational fractional coordinates. The certifier:

1. normalizes positive tetrahedron orientation and rejects degeneracy;
2. generates complete periodic AABB candidates through `_periodic_spatial.py`;
3. classifies exact tetrahedron pairs using face-normal and edge-cross-edge separating axes;
4. permits only disjoint interiors or exact boundary contact;
5. requires every periodic triangular facet orbit to have exactly two opposite incidences;
6. classifies pairs as auxiliary-internal or scientific interfaces after applying the exact inter-image translation;
7. requires every scientific interface triangle to match exactly one selected face-witness triangle orbit;
8. requires every complete tile side to cover all triangles of that witness once with one orientation;
9. reconstructs the scientific shell and requires exact equality with ∂_3; and
10. after all topological and nonoverlap gates, requires positive exact tile volumes summing to one primitive fractional domain.

Tile overlap means

$$\operatorname{int}(T_i) \cap \operatorname{int}(T_j) \neq \varnothing.$$

Prescribed shared faces, edges, or vertices are allowed. Partial interpenetration, full containment, and coincident interiors are distinguished and rejected. Volume closure is never used alone as a no-void/no-overlap proof.

The overlap predicate adapts the tetrahedron separating-axis framework of Ganovelli, Ponchio, and Rocchini to exact `Fraction` projections. Exact-sign semantics follow Shewchuk. Translation-labelled quotient incidence follows the periodic vector-graph representation of Chung, Hahn, and Klee. These sources are cited in the module and Stage-9 specification; the cellular shell and certificate composition are the project-specific construction.

Source-replay deserialization reconstructs the scientific complex from proposed shells and reconstructs the partition certificate from auxiliary vertices and tetrahedra. Serialized boundary matrices, overlap counts, facet pairs, coverage, and volumes are not trusted as independent truth.

`natural_tiling.py`

`natural_tiling.py` orchestrates ring-bound refinement, symmetry, strength, embedded-face constraints, periodic partition construction, and the scientific natural-tiling rules.

The solver is proper relative to one explicit `PeriodicNetView`:

$$\operatorname{Aut}(\mathcal{T}) = \operatorname{Aut}(G_{\text{view}}).$$

Approximate Cartesian symmetry of an MD frame is not used. Properness is certified on the **scientific tiling**: face placements, translation-labelled attaching maps, and tile orbits must be invariant under the finite automorphism representatives modulo translation. Auxiliary face triangulations and partition meshes need only certify that scientific structure; they are not required to have identical symmetry combinatorics.

The solver follows Blatov et al. [9]: preserve net symmetry, use locally strong ring faces, split along admissible non-face strong rings, and resolve published crossing alternatives. It prunes by symmetry and compatibility before cell construction; it does not enumerate every arbitrary tiling first.

The primitive-ring bound is an orchestration boundary. Increasing K triggers

```
rebuild primitive rings
-> rebuild source-bound indexes
-> recompute induced ring symmetry
-> recompute strength domains/results
-> recompute scientific faces/witness constraints
-> reconstruct and revalidate cell complexes
```

No downstream object identified only by dense local IDs is reused across the refinement. Stable ring keys and net-view digests are used to compare revisions and report which scientific results changed.

The scientific outcome and the certification report are separate because the conditions are not mutually exclusive. Conceptually:

```
NaturalTilingOutcome(
    tilings=...,
    outcome=UNIQUE | MULTIPLE | NONE,
)

NaturalTilingCertification(
    primitive_ring_bound=...,
    primitive_complete=...,
    strength_domains=...,
    strength_complete=...,
    embedding_complete=...,
    compatibility_complete=...,
    partition_certified=...,
    properness_certified=...,
    resource_truncations=...,
    unresolved_witnesses=...,
)
```

A result may therefore be simultaneously ambiguous, strength-bounded, embedding- conditional, and partition-certified. No unresolved assumption is collapsed into a single status label.

Natural-tiling strength analysis requires a lower-closed primitive-ring catalog from the minimum supported cycle size through the active bound K. A catalog containing only a requested upper interval, for example sizes 6–8, is insufficient to certify strength of an 8-ring because smaller decomposition components may be absent.

Only ring orbits used as faces of an accepted result are called **essential**. Multiple surviving certified or conditional tilings remain explicit; enumeration order is never a scientific tie-breaker.

`natural_tiling_search.py`

`natural_tiling_search.py` implements the first complete finite generator relative to an exact Stage-9 master refinement. Every master scientific interface is an available cut. The complete Stage-10A face action partitions those interfaces into full net-symmetry orbits, and the search enumerates unions of boundedly strong orbits only. Hard and unresolved witness constraints are tested before geometric reconstruction.

For a selected cut set, auxiliary-internal facets and omitted scientific interfaces define a translation-labelled quotient graph on master tetrahedron orbits. Propagated image shifts satisfy

$$\mathbf{s}_j = \mathbf{s}_i + \boldsymbol{\tau}_{ij}.$$

If one quotient tetrahedron receives two different shifts, the corresponding closed walk carries nonzero lattice translation and its lifted component is periodically unbounded. Otherwise each quotient component lifts to translates of one finite tetrahedral tile interior. Retained interfaces reconstruct exact oriented face terms relative to those translated tile placements.

Every generated shell is passed through `build_periodic_cell_complex()`, and the master tetrahedra are re-certified by `certify_periodic_tetrahedral_partition()` after only their scientific tile assignments are changed. The final finite rule retains every viable selection not strictly contained in another viable selection. This realizes splitting along every admissible strong-ring cut inside the declared master arrangement while preserving incomparable crossing alternatives.

Search completeness is independent of candidate eligibility. Missing or truncated strength and unresolved compatibility remain `UNRESOLVED`; individually eligible Stage-10A candidates are conditional until the Stage-10B finite family is complete. Alternative disk witnesses require separate master refinements because the auxiliary tetrahedral arrangement may conform to only one witness assignment.

`ring_geometry.py`

Responsibility

`ring_geometry.py` computes frame-dependent descriptors for persistent ring identities: lifted coordinates, centers, normals, area vectors, perimeter, planarity, puckering, aperture, and distortion.

This is a parallel physical-analysis branch after `PrimitiveRingCatalog`; it is not a mandatory predecessor of strong-ring classification or topological symmetry. Geometry is evaluated only on frames compatible with the owning topology class.

`ring_site.py`

Responsibility

`ring_site.py` constructs species-dependent physical site hypotheses and certified microstates from persistent ring geometry, oriented ring–tile side anchors, and framework semantic profiles. It does not assume one site per ring or one site per side. A ring may support no bound state, one side-localized state, a bilateral double well, one plane-centered state, several off-center angular states, an annular state, or an unresolved general landscape.

Scientific ring and side-anchor identities remain species-independent. Physical site identity includes the target species, landscape model, local-state key, and provenance. Occupancy and observed transitions are dynamic observations and do not mutate either the topological ring catalog or the natural tiling.

`tiling_geometry.py`

Responsibility

`tiling_geometry.py` realizes a persistent `NaturalTilingCatalog` in compatible frames. It computes tile centers, oriented face geometry, volumes, deformation, and dual-network geometry without changing tile incidence or identity.

`cage.py`

Responsibility

`cage.py` adds physical and chemical interpretation to natural tiles and shared faces. It distinguishes

```
NaturalTile
TopologicalWindow
AccessibleCage
AccessiblePortal
```

Guest-excluded volume, aperture, guest radius, containment, and conservative void/portal witness certificates belong here. Conventional framework labels are owned by the semantic registry. Species-dependent microstates, transition pathways, and kinetic rates belong to the site-state kinetic branch. Topological tile identity, face incidence, and properness remain owned by the tiling modules.

Identity, fingerprints, and reproducibility

Exact labeled equality

Within one compatible atom-indexed collection and one mapping, topology equality is exact equality of canonical labeled decorated edge records. General graph isomorphism and automatic equivalence between different atom orderings, primitive cells, supercells, or lattice bases are outside the first scope.

Structures with different atom orderings require an explicit atom-identity mapping before topology or ring identities can be compared.

Canonical schema and stable digests

Canonical topology and ring representations must carry a schema version. Persistent fingerprints should use a stable digest such as SHA-256 or BLAKE2 over deterministic serialized records, not Python's process-dependent built-in `hash()`.

The package must distinguish:

- a structured canonical key, which defines equality;
- a stable digest, which is a compact identifier;
- a dense local or global integer ID, which is a convenient catalog index.

Immutability

Identity-bearing objects, including framework mappings, connectivity definitions, connectivity states, framework topologies, primitive rings, and ring catalogs, should be immutable after construction. Derived arrays should be read-only where practical. A transformation creates a new object rather than invalidating an existing fingerprint.

Periodic gauge normalization

Periodic edge translations have a gauge freedom. If vertex image representatives change by integer shifts $\mathbf{g}_i$, then

$$\mathbf{m}'_{ij} = \mathbf{m}_{ij} + \mathbf{g}_j - \mathbf{g}_i.$$

The physical periodic graph is unchanged. `framework_topology.py` chooses a deterministic gauge per connected component before edge translations enter canonical keys.

A `PrimitiveRingKey` is stable **after** this deterministic framework gauge has been selected. The ring key is not independently invariant under arbitrary changes of vertex representatives, atom indexing, lattice basis, primitive-cell choice, or supercell choice. Comparisons across such representations require an explicit periodic-net mapping.

Atom identity contract

Persistent topology and ring labels assume that atom index i refers to the same physical atom in every frame being compared.

A compatible collection must preserve:

- atom count;
- per-index atomic number;
- atom identity and ordering;
- periodic-dimensionality convention.

Equal composition alone is insufficient. Separately generated files with different atom orderings require an explicit atom-identity mapping before persistent topology or ring labels are meaningful.

Topology fingerprint

A topology fingerprint should depend on:

- framework mapping definition;
- retained atom identities and species;
- canonical projected edge-path keys;
- periodic edge translations;
- relevant mapping options;
- atomic connectivity definition identity;

- canonical-schema version.

It should not depend on:

- graph iteration order;
- Cartesian rotation or translation;
- arbitrary NetworkX node insertion order;
- frame order.

Ring canonical key

A `PrimitiveRingKey` is the stable topology-local identity of a ring orbit. In the current implementation it is the normalized cyclic sequence of complete decorated framework-edge tokens after the owning `FrameworkTopology` has fixed its deterministic periodic gauge. The canonical step sequence and reconstructed lifted vertex walk are deterministic associated records; the vertices are not separately stored inside the key.

The key is normalized under:

- cyclic rotation;
- reversal; and
- translation of the complete lifted ring.

Dense `ring_id` values are catalog-local conveniences. They may change when the ring-size bound is enlarged because newly discovered keys can alter deterministic ordering. The digest of a `PrimitiveRing` may also include catalog-local or search-provenance fields and is therefore not the preferred identity across bound refinements.

Persistent downstream ring references use

$$(\text{topology graph digest}, \texttt{PrimitiveRingKey}).$$

Symmetry and tiling results additionally record the `PeriodicNetView` digest and, for embedded results, the `PeriodicNetEmbedding` digest. These are compatibility and provenance identities layered on top of the source ring identity rather than replacements for it.

A source-bound index resolves the key to a local integer when efficient access is required.

Global ring catalog across topologies

Across topology classes, exact key equality is meaningful only when the source framework representations and canonical gauges are comparable. Different atom orders, cell bases, primitive/supercell choices, or net-view policies require an explicit mapping before keys are compared.

Within one topology and net view, increasing the primitive-ring bound preserves key meaning but may change dense IDs and all downstream catalogs. The pipeline therefore rebuilds dependent results and compares them by stable keys.

Across genuine topology changes, a project-level lineage record may report that a stable mapped ring key is present, absent, or reappears. Approximate lineage based on overlap is a separate inferred relation and must not be presented as exact identity.

Cross-cutting provenance

Every major result should preserve enough information to reproduce its scientific definition:

- source frame IDs and collection identity;
- atomic connectivity definition and thresholds;
- cutoff provenance;
- framework mapping;
- topology and canonical-schema versions;
- algorithm names and options;
- safety limits and completeness flags;
- topology and ring digests;
- active `PeriodicNetView` policy/digest when symmetry or tiling is involved;
- active `PeriodicNetEmbedding` digest and projected-edge model for embedded results;

- periodic-spatial broad-phase request, grid/stencil diagnostics, and cache rebuild provenance when geometric candidate caching is used;
- package version.

A ring catalog without its connectivity model and framework mapping is incomplete scientific provenance.

Standard analysis workflows

Nonreactive trajectory

For a zeolite, the connectivity scope may be restricted to Si, Al, and O, or a broader connectivity graph may be used with Na, Li, and K assigned as spectators in the framework mapping.

```
trajectory frames
      |
      v
classify atomic connectivity with trajectory-safe rules
      |
      v
build reference framework topology
      |
      v
validate topology across all frames
      |
      v
TopologyConsistency.UNIFORM
      |
      v
find primitive rings once
      |
      v
compute per-frame ring geometry
      |
      v
site occupancy and transitions
```

Fixed-topology ensemble

```
independent frames of the same framework
      |
      v
classify connectivity using reference or ensemble-safe rules
      |
      v
classify topology without using time order
      |
      v
TopologyConsistency.UNIFORM
      |
      v
one deterministic ring catalog
      |
      v
geometry and site statistics over frames
```

Cells and volumes may vary while topology and atom identity remain fixed.

Partitioned reactive trajectory

```
trajectory
   |
   v
hysteretic or explicit atomic connectivity states
```

```
   |
   v
projected topology fingerprints
   |
   v
A A A A B B B B
   |
   v
TopologyConsistency.PARTITIONED
   |
   +--> topology A ring catalog
   |
   +--> topology B ring catalog
   |
   +--> transition A -> B with changed edges
   |
   v
stable mapped ring keys and affected-ring analysis
```

Ring search is performed once per unique topology class, not once per frame.

Multi-topology ensemble

```
unordered independent frames
      |
      v
frame-local, reference, or consensus connectivity
      |
      v
group by topology class
      |
      v
frames_by_topology[A], frames_by_topology[B], ...
```

No temporal transition is inferred from adjacent frame indices.

Per-frame topology analysis

When topology changes too frequently or a reference topology is inappropriate, each frame may be analyzed independently. Repeated topology hashes may still be reused internally, but persistent site or ring trajectories are not assumed unless exact canonical identities can be reconciled.

Heterogeneous solid-liquid or solid-solid interface

```
persistent atom groups and initial material identities
      |
      +--> fixed ConnectivityScope for structural connectivity
      |
      v
atomic connectivity and topology catalogs
      |
      +--> broken bonds, detached components, and ring changes
      |
      v
region_membership.py
      |-- align anchors or tracked backbones
      |-- estimate current material boundaries
      |-- classify geometric region
      `-- classify attached versus detached original atoms
      |
      v
ring-site, adsorption, dissolution, and transfer analysis
```

An atom leaving a slab remains in the persistent structural atom set. Its changed connectivity and topology are recorded first; region membership then reports that it has moved into another phase or become detached. Conversely, a spectator ion may enter the geometric solid region without becoming part of the framework.

Validation strategy

Validation proceeds from exact source identity and periodic multigraph invariants to bounded natural-tiling certificates.

Implemented-layer regression

The implemented atomic-connectivity, framework-topology, topology-catalog, and primitive-ring suites remain the baseline. Any later private helper extraction must preserve public imports, JSON payloads, digests, deterministic ring keys, and the established LTA count

$$36 \times 4\mathrm{R} + 40 \times 6\mathrm{R} + 6 \times 8\mathrm{R}.$$

Primitive-ring proof fixtures continue to cover translation-orbit completeness, even/odd reconstruction, finite reduction H_K, parallel edges, and resource truncation.

Net-view tests

- deterministic `PeriodicNetView` digest;
- exact source vertex/edge mappings;
- no collapse of parallel edges when labels are ignored;
- distinct unlabeled-T and chemically decorated policies;
- connected-component count;
- cycle-gain translation subgroup and rank; and
- explicit rejection of non-three-periodic inputs by the first tiling backend.

Stage-5 prototype tests

- stable-key ring placement independent of dense local ID;
- exact forward/reverse parametrization using the stated vertex/edge formulas;
- unique physical placement despite $2n$ boundary parametrizations;
- no spurious cancellation between translated edge instances;
- translated placements covering a requested physical edge;
- private ring-occurrence mapping under identity, translation, reversal, and repeated edge-token symmetries; and
- complete invalidation/rebuild behavior when the ring bound changes.

Symmetry tests

- finite representatives modulo translation;
- deterministic shift gauge;
- exact group identity, inverse, composition, and closure;
- explicit vertex and edge permutations in a periodic multigraph;
- preservation of endpoint incidence and policy signatures;
- exact action on lifted edge instances;
- explicit ring vertex/step occurrence maps and stabilizers;
- composition of induced ring actions; and
- robustness to arbitrary coordinate perturbation preserving topology.

Strength tests

- proof fixtures showing smaller primitive components suffice;
- immutable mathematical `RingStrengthDomain`;
- resource limits separated from the domain;
- exact weak-ring witnesses over physical edge instances;
- exhaustive `STRONG_IN_DOMAIN` certification;
- separate incomplete-source and truncated-search statuses;
- symmetry transport by stable ring key; and

- rebuild comparison under increased primitive and strength bounds.

Stage-7R boundary tests

Test the persistence split explicitly:

- core net symmetry serializes without primitive-ring data;
- primitive-ring symmetry is bound to both symmetry and exact ring-catalog digests;
- compact ring-position actions reproduce key-level orbits and cocycle-corrected placement maps;
- barycentric placement is exact, source-bound, collision-diagnostic, and resource-bounded;
- strength results exclude candidate workspaces;
- weak witnesses and bounded negative results are independently reverified on load; and
- matrix/provenance resource exhaustion returns unresolved status.

Periodic-net-embedding tests

- exact view/discovery/embedding compatibility and digest mismatch rejection;
- exact edge-covariance metric and primitive integral normalization;
- metric covariance under a nontrivial unimodular lattice-basis shear;
- exact affine equivariance of every embedded vertex and straight edge under the complete discovered symmetry group;
- unit-volume Cartesian cell and lifted straight-segment endpoint checks;
- rejection of collided vertices, singular metrics, zero-length edges, and coincident distinct straight projected edges;
- transactional vertex, edge, symmetry-order, and rational-bit resource failure;
- independence of the authoritative embedding from distorted trajectory coordinates; and
- Na-LTA construction under the complete 96-operation net symmetry.

Periodic spatial broad-phase and cache tests

- continuous lifted supports crossing periodic boundaries produce the same exact candidates as exhaustive lifted-image enumeration;
- direct translation-stencil candidates equal exhaustive image enumeration on small orthogonal and skewed cells;
- automatic subcell selection never changes candidate completeness;
- multi-bin occupancy retains image labels, generates no missed pairs, and canonical deduplication removes repeats without collapsing valid self-images;
- large-cell linked-cell results equal direct stencil results exactly;
- fixed image shells are not used as correctness assumptions;
- distance, intersection, penetration, linking-support, and filled-volume overlap queries each use a conservative support rule and agree with exhaustive exact predicates on finite fixtures;
- object-distance change obeys the vertex-displacement bound on translating, rotating, bending, and deforming PL fixtures;
- multi-image cached candidates equal fresh extended-cell-list candidates while the Verlet margin is positive;
- the shared validity kernel does not assume MIC or one image per object pair;
- topology/mesh/witness/support-rule/request changes force rebuild; and
- deformation-aware object caches equal fresh rebuilds under strain, shear, and rigid cell rotation.

Embedded-face tests

- explicit reference-embedding provenance;
- planar and nonplanar disk certificates;
- multiple embedding witnesses for one scientific face placement without duplicate face identity;
- symmetry action on scientific faces with equivalent-but-not-identical witnesses;
- self-intersection, framework penetration, periodic intersections, and linking;
- nonzero algebraic ring–surface intersection as a linking certificate;
- intersecting witness disks classified as witness incompatibility rather than automatic catenation;
- disjoint embedded disk witnesses as an unlinking certificate in supported cases;
- unary, pairwise, higher-order, and symmetry-linked compatibility constraints;
- robust near-degenerate predicates; and
- propagation of unresolved strength/embedding assumptions.

Periodic-cell-complex tests

- translation-labelled attaching maps;
- oriented boundary operators and $\partial_1\partial_2 = \partial_2\partial_3 = 0$;
- self-neighbor and self-image incidence;
- edge and vertex links;
- connected, orientable, nonbranching genus-zero tile boundaries;
- quotient Euler relation $N_0 - N_1 + N_2 - N_3 = 0$;
- exact periodic closure;
- valid shared-boundary contact versus improper crossing versus interior overlap;
- nested-volume containment overlap even when boundaries do not intersect; and
- an explicit no-overlap/no-void partition certificate, with volume closure used only as a diagnostic.

Natural-tiling tests

- properness relative to the exact `PeriodicNetView` group;
- complete scientific face/tile orbit invariance independent of auxiliary witness triangulation;
- local-strength split rules;
- certification-state propagation;
- ambiguity preservation;
- full downstream invalidation after a ring-bound increase; and
- stable-key refinement reports.

LTA domain validation

For the unlabeled LTA T-net, the end-to-end target remains

$$[4^6], \qquad [4^{6.6^8}], \qquad [4^{12}.6^{8.8^6}],$$

with tile multiplicity ratio

$$3 : 1 : 1.$$

The accepted essential ring types must agree with the published LTA natural tiling. Results should remain stable when the primitive bound is increased from 8 to 10 and 12, while dense ring IDs are allowed to change.

Implementation sequence

Stages 1-4R - implemented

Atomic connectivity, framework topology, topology catalogs, topology statistics, and the complete bounded primitive-ring algorithm are implemented. The bounded periodic correctness proof is part of the Stage-4R contract.

Stage 5-P0/P1 - exact ring-placement infrastructure - implemented

Implemented in `mdstats 0.19.10a0`:

- structural validation that stored canonical `vertex_walk` records are continuous with their oriented quotient-edge steps;
- stable `PrimitiveRingKey` lookup against one exact source catalog;
- orientation-independent physical edge-instance anchors;
- occurrence-level edge-orbit inverse incidence;
- exact translated ring placements covering a requested lifted edge instance;
- Na-LTA gate: 82 ring orbits and 432 canonical ring-step occurrences.

The source primitive-ring keys, ring digests, and catalog digest are unchanged from revision 16.

Stage 5-P2 - exact automorphism-induced ring occurrence action - implemented

Implemented in `mdstats 0.19.11a0` by `periodic_ring_action.py`:

- exact lifted vertex action `(i,n) -> (pi(i), A n + tau_i)` with `A in GL(3,Z)`;
- explicit edge permutation and image orientation for periodic multigraphs;
- exact validation of quotient-edge endpoint and image-shift consistency;

- exact lifted physical edge-instance mapping;
- stable-key target ring lookup after transforming the ordered edge occurrence sequence;
- authoritative source-vertex and source-step occurrence permutations;
- exact cyclic/reversed alignment verified by lifted vertices and physical edge instances;
- Na-LTA translated-identity gate over all 82 ring orbits and 432 ordered steps.

The original P2 module was a validation/application prototype only. Revision 22 retains its exact ring-occurrence machinery and enforces one explicit `PeriodicNetView` signature policy and view digest. Revision 23 normalizes representatives and assembles exact finite generated groups in Stage 6B. Revision 24 supplies the complete automatic Stage-6C discovery backend and the exact translation cocycle required by absolute lifted placements.

Stage 5-P3 - exact finite GF(2) primitive-ring support cancellation - implemented

Implemented in `mdstats 0.19.12a0` by `primitive_ring_cancellation.py`:

- exact `RingPlacementSupport` over physical `LiftedEdgeInstanceRef` basis elements;
- exact translation covariance of support anchors;
- deterministic finite $\mathrm{GF}(2)$ span membership using temporary integer bitsets;
- strict smaller-ring candidate validation aligned with the strong-ring definition;
- exact positive `RingCancellationWitness` with independent physical-edge cancellation verification;
- exact negative status `NOT_IN_SUPPLIED_SPAN` with no strength promotion;
- synthetic weak-primitive fixture where one 6-ring is the symmetric difference of three 4-rings;
- wrong-periodic-image and incomplete-candidate controls preventing false cancellation; and
- Na-LTA support verification over all 82 represented ring orbits.

The P3 module does not enumerate a `RingStrengthDomain`, prove a global finite placement bound, classify strength, or expose public chain algebra. Those remain Stage 7 responsibilities.

Stage 5-R - API hygiene and shared periodic infrastructure - implemented

Implemented in `mdstats 0.19.13a0` after comparing P1-P3:

- source-bound `RingPlacement(topology_graph_digest, ring_key, image_shift)`;
- source-bound `LiftedEdgeInstanceRef(topology_graph_digest, edge_key, anchor_shift)`;
- private `_periodic_graph.py` arithmetic shared only where duplication was proven;
- supported `canonicalize_primitive_ring_tokens()` instead of cross-module import of a private helper;
- `CycleParameterization` with exact vertex/step permutation formulas;
- ordered canonical and translated physical-edge support accessors on `PrimitiveRingIndex`;
- hidden internal occurrence buckets and removal of the occurrence record from the public analysis API;
- advanced Stage-5 infrastructure retained under `mdstats.analysis` rather than the package root; and
- full package regression coverage: `586 passed, 28 warnings` across three nonoverlapping test-file groups.

No primitive-ring catalog digest, ring enumeration rule, or scientific result is changed by this cleanup. `PeriodicNetView` and the revision-22 view-bound action layer now build on this frozen infrastructure.

Stage 5.1 - explicit net-view contract (implemented)

`mdstats 0.19.14a0` implements the first `PeriodicNetView` backend as a signature projection of the exact `FrameworkTopology` vertex/edge orbit sets. The module provides:

- deterministic enum-based `NetViewPolicy` vertex/edge signatures;
- unlabeled-framework and chemically decorated built-in policies;
- exact source atom/edge-key mappings without graph copying or contraction;
- explicit preservation of parallel-edge multiplicity even under equal signatures;
- source graph, source topology, policy, and view digests;
- deterministic quotient components and fundamental cycle-gain generators;
- exact translation rank and full-rank subgroup index; and
- a strict `natural_tiling_eligible` precondition requiring full 3D PBC, one quotient component, rank three, and index one.

Structural filtering/contraction remains deferred because it would define a new graph source and require a compatible primitive-ring catalog. Revision 22 now binds validated explicit automorphism records to the net-view digest; automatic symmetry discovery and group-catalog ownership remain the next stages.

Stage 5.2 - private consumer prototypes

Before any general refactor, prototype directly against the current `PrimitiveRingCatalog`:

1. automorphism-induced mapping of every ordered ring vertex and step occurrence (**implemented and retained as a regression gate**);
2. translated primitive-ring placements covering exact physical edge instances (**implemented and retained as a regression gate**); and
3. exact finite modulo-two cancellation for small known decompositions (**implemented and retained as a regression gate**).

All three consumer prototypes pass and their concrete duplication has now been reviewed. Revision 20 extracts only the justified private lattice/edge-instance arithmetic and freezes source-safe placement/parametrization contracts; broader periodic helper extraction remains prohibited without another concrete consumer.

Stage 5.3 - stable identity and lightweight views

Source-safe `RingPlacement`, separate `CycleParameterization`, and source-bound `PrimitiveRingIndex` are now implemented. Introduce lazy `OrientedRingView` only when a geometry consumer requires it, and retain exact private edge support. Do not introduce a public chain algebra or global ring-contact catalog.

Stage 5.4 - extract shared private helpers and freeze API

Only after the prototypes pass, extract repeated lattice/edge-instance operations into `_periodic_graph.py` or another minimal helper. Freeze Stage-5 names only when both symmetry and strength consumers can be implemented without additional identity or orientation workarounds.

Stage 6A - view-bound explicit automorphism validation (implemented)

`mdstats 0.19.15a0` upgrades the Stage-5 P2 action prototype so that every validated operation belongs to one exact `PeriodicNetView.digest`. The builder:

- validates integer unimodular lifted-vertex action;
- requires explicit multiedge permutation and orientation;
- enforces vertex and edge signatures from the active `NetViewPolicy`;
- rejects lattice matrices that mix active periodic translations into nonperiodic axes;
- validates exact quotient-edge endpoint/image-shift incidence;
- maps physical edge instances using view edge positions; and
- bridges the view and primitive-ring edge domains only through stable `FrameworkEdgeKey` before inducing exact `RingOccurrenceMap` records.

An action validated under one view cannot be reused under another view of the same topology. Revision 23 builds the finite generated group on top of this validator; revision 24 adds complete automatic generator discovery for the exact first-backend domain.

Stage 6B - explicit-generator periodic symmetry group - implemented and hardened

Assemble finite closure modulo translations from validated generators. Store the normalized group, multiplication/inverse tables, exact composition-translation cocycle, and vertex/edge orbits. The core object owns no ring-derived data.

Stage 6C - automatic periodic-net symmetry discovery - implemented

Use an exact rational barycentric placement and deterministic source-star frame to discover every automorphism in the supported stable three-periodic domain. Build the core group and, when requested, a separate primitive-ring symmetry index. Unsupported, collision-degenerate, flat-star, and resource-truncated cases fail transactionally.

Stage 7 - bounded strong-ring classification - implemented

Enumerate an explicit target-connected incidence-depth domain of translated smaller primitive rings. Solve exact physical-edge support membership over $\mathrm{GF}(2)$ and return a positive witness, bounded negative theorem, or unresolved source/resource status.

Stage 7R - certification and persistence consolidation - implemented

1. Split core `PeriodicNetSymmetry` from catalog-bound `PrimitiveRingSymmetryIndex`.
2. Extract reusable source-bound `PeriodicBarycentricPlacement`.
3. Separate persistent `RingStrengthResult` from transient `RingStrengthSearchWorkspace`.
4. Independently verify strength certificates during deserialization.
5. Add explicit matrix/provenance bit-resource guards to finite cancellation.
6. Replace repeated linear source lookups with transient immutable maps.
7. Freeze schema versions for the cleaned ownership boundary.

This stage changes derived persistence schemas but does not change primitive-ring enumeration, exact symmetry groups, ring orbits, or bounded strength mathematics.

Stage 8A - authoritative periodic-net embedding - implemented

Construct the collision-free exact barycentric realization, derive the basis-covariant edge-covariance lattice metric, verify complete symmetry equivariance, expose the unit-volume Cartesian cell and straight lifted edge segments, and persist source-bound provenance. Global periodic edge-crossing certification is supplied by the implemented Stage 8B certificate.

Stage 8B - periodic extended-object broad phase - implemented

Implemented `_periodic_spatial.py` with continuous lifted fractional AABB supports, a complete support-derived translation stencil, direct and automatically selected linked-cell candidate generation, conservative multi-bin occupancy, explicit nonzero self images, deterministic `(object_i, object_j, image_shift)` canonicalization, and transactional object/image/check/insertion resources. Direct and linked-cell outputs are regression-tested for exact equality.

Implemented `periodic_edge_intersection.py` with exact rational segment predicates and a source-bound `PeriodicEdgeIntersectionCertificate`. Allowed shared lifted vertices are distinguished from proper crossings, endpoint-on-interior contacts, distinct-vertex endpoint collisions, and collinear overlaps. The Na-LTA straight-edge embedding is globally certified intersection-free.

Deformation-aware extended-object cache reuse remains deferred. The existing Verlet validity theorem is still the planned mathematical kernel, but no cache API is frozen until a time-dependent extended-object consumer exists. Segment–triangle, triangle–triangle, penetration, linking, and filled-volume predicates remain Stage 8C/9 consumers of the implemented query-agnostic broad phase.

Stage 8C - embedded face placements - implemented

Implemented `_robust_geometry.py` with exact rational segment–triangle and triangle–triangle predicates, including coplanar clipping and explicit point, segment, and area degeneracies. Implemented `_surface_mesh.py` with deterministic exhaustion of all oriented boundary-vertex triangulations and exact Catalan resource preflight.

Implemented `face_candidates.py` with mesh-independent scientific `FacePlacement`, auxiliary `FaceEmbeddingWitness`, exact periodic disk self-intersection rejection, framework-penetration certificates, nonzero algebraic ring–surface linking certificates, particular-witness incompatibility and disjoint disk semantics, prescribed shared-boundary handling, source-replay serialization, symmetry mapping, and finite compatibility constraints including caller-declared higher-order forbidden tuples.

The first backend is complete only for its declared boundary-vertex triangulation family. Failure to find an embedded or admissible disk remains `UNRESOLVED`; no knottedness or general link theorem is inferred. Natural-tiling face selection remains a Stage-10 responsibility; Stage-9 cell-complex validation is implemented.

Stage 9 - periodic cell complex and partition certificate - implemented

Implemented translation-labelled integer boundary operators, exact chain composition, caller-supplied tile-shell validation, quotient and lifted-shell Euler invariants, and source-replay persistence. Implemented a separate exact periodic tetrahedral

certificate with complete periodic broad-phase candidates, rational interior-overlap classification, opposite-oriented facet pairing, exact selected- witness conformity, induced-shell equality, and exact unit-domain volume closure. Local face-sector propagation remains deferred because no complete first-backend theorem has been established for it.

Stage 10 - natural-tiling orchestration

Stage 10A - candidate and properness certification - implemented

Implemented `natural_tiling.py` with exact scientific face/tile actions under the complete Stage-7R automorphism group, normalized group-composition validation, proper/improper classification, multidimensional candidate certification, scientific/evidence digest separation, essential-ring extraction, and explicit `NONE`, `UNIQUE`, or `MULTIPLE` catalog outcomes. The implementation validates caller-proposed Stage-9 complexes only; it does not infer a natural tiling.

Stage 10B - natural face selection and local splitting - implemented

Implemented `natural_tiling_search.py` over one exact caller-supplied master refinement. The first backend computes full-symmetry master-face orbits, selects only `STRONG_IN_DOMAIN` orbits, preflights and exhausts every nonempty orbit subset, and prunes hard or unresolved fixed-witness compatibility before exact construction. Removing an interface adds its translation-labelled tetrahedral adjacency. A repeated quotient tetrahedron with an inconsistent propagated image shift certifies a nonzero translation cycle and rejects the resulting noncompact slab or channel.

Finite components are converted to translated tile placements; retained face sides must cover every master witness triangle exactly once with one orientation. The derived shells are validated by Stage 9, the unchanged tetrahedral geometry is re-certified under the new tile assignment, and Stage 10A rechecks properness and eligibility. Only inclusion-maximal viable strong-ring selections survive; incomparable alternatives remain `MULTIPLE`. Search completeness is separate from the Stage-10A catalog, so unresolved strength or compatibility makes the result conditional. The backend does not infer the master refinement or enumerate alternative witness geometries within one run.

Stage 10C - ring-bound refinement - implemented

Implemented `natural_tiling_refinement.py`. For every requested upper bound K, the caller supplies one independent complete downstream rebuild. `PrimitiveBoundBuild` verifies that the ring index, induced symmetry, bounded strength, face certificates, compatibility systems, master complexes, partition certificates, Stage-10B searches, and aggregate Stage-10A catalog all name the current primitive-ring catalog. An earlier-bound object cannot be reused merely because its dense IDs still fit.

`build_primitive_bound_snapshot()` removes catalog-local storage identity and produces stable category records for rings, ring orbits, strength results, scientific faces, compatibility systems, master complexes, master partitions, searches, natural tilings, and essential rings. A face key is the exact tuple of embedding digest, primitive-ring key, image shift, and orientation. A stable complex rewrites ∂_2 and tile-shell incidence through those face keys and canonically sorts tile representatives.

Consecutive bounds are compared by `(category, stable key)` and every scientific addition, removal, or state modification is retained. For complete lower-closed primitive searches,

$$\mathcal{R}_K \subseteq \mathcal{R}_{K'} \qquad (K < K'),$$

so disappearance of a primitive-ring stable key is `INVALID`. Any incomplete endpoint makes the transition `UNRESOLVED`; complete changed endpoints are `CHANGED`; only exact stable-record and outcome equality is `STABLE`. `stable_tested_suffix_start` reports the earliest bound in the final consecutive stable suffix of the supplied sequence. It is explicitly not an asymptotic convergence proof.

Stage 10D - LTA end-to-end gate - implemented

Implemented `lta_natural_tiling.py`. The strict first backend accepts the exact unlabeled LTA quotient fingerprint, discovers the complete order-96 group and authoritative embedding, and rebuilds primitive rings, depth-one strength, and exact polygon geometry at $K = 8, 10, 12$. A ring becomes a scientific face only when it is both `STRONG_IN_DOMAIN` and strictly convex planar. The selected set is therefore 36 four-rings, 16 six-rings, and 6 eight-rings at every tested bound; the 32 new strong twelve-rings at $K = 12$ are retained as nonplanar exclusions.

Canonical convex fan witnesses pass exact periodic self-intersection and framework penetration tests. Around every lifted framework edge, exact quotient-plane ray ordering joins consecutive oriented face sides with explicit translation gains. Zero-gain components produce ten finite shells and the Stage-9 complex $(48, 96, 58, 10)$. The recovered tile multiplicities are

$$6[4^6] + 2[4^{6.6^8}] + 2[4^{12}.6^{8.8^6}],$$

which reduce to $3 : 1 : 1$. Properness follows because every exact generator preserves the scientific faces and shells and the stored multiplication table proves those generators close to all 96 net automorphisms.

The LTA-specific partition certificate proves each shell is a convex polytope with a strict rational interior point, computes positive exact volumes summing to one, and excludes every periodic interior overlap using the complete convex-polytope separating-axis family of face normals and edge-direction cross products. Higher- bound downstream evidence is reused only after exact selected-ring stable-key equality is established from independent ring, strength, and geometry rebuilds. This is a certified LTA ground gate, not a generic automatic master-refinement algorithm.

Stage 11 structural completion and Part II handoff

Stage 11 is partitioned by scientific ownership rather than by chronology. Part I ends after the species-independent structural layers are complete.

Part I implemented structural stages

- **Stage 11A:** exact reference tile geometry, topological windows, conservative cage/portal witnesses, and periodic accessibility rank;
- **Stage 11B:** compatible-frame natural-tile geometry without rediscovering topology;
- **Stage 11C1:** persistent reference T/O ring geometry;
- **Stage 11C2:** compatible-frame ring centers, normals, side frames, and breathing descriptors;
- **Stage 11C3:** atom-resolved T/O chemistry, exact cyclic sequences, serrated oxygen boundaries, harmonic descriptors, and dihedral gauges; and
- **Stage 11D:** framework semantic registries and explicit LTA convention profiles.

These stages export stable ring, tile, cage, window, atom/image, translation, and semantic identities. They do not create a mobile-ion site, transition, barrier, rate, or kinetic state.

Normative handoff contract

Part II consumes only source-bound Part I products:

```
certified framework topology
    -> primitive-ring and natural-tiling identities
    -> reference or registered structural geometry
    -> framework semantics and serrated ring descriptors
    -> Stage C0 registered trajectory evidence
    -> species-dependent statistical-site analysis
```

The following rules are permanent:

1. structural identity is species-independent;
2. the atom-resolved ordered oxygen polygon is the scientific ring boundary;
3. structural rings/windows/cages are candidate interpretation objects, not automatically physical sites;
4. topological adjacency is not automatically an observed transition;
5. a geometric bottleneck is not automatically a barrier or rate; and
6. Part II may reject, leave unresolved, or classify a structural association without mutating Part I identities.

Part II ownership

The complete plan and status for Stage C0, Stage 11E, the E8a pilot, E8b/E9, and deferred Stages 11F-I are maintained only in `stage11_site_kinetics_architecture.{md,pdf}`. This manual intentionally does not carry a second copy of those plans.

Deferred features

The following ideas remain compatible with the architecture but are intentionally deferred:

- an unbounded global strong-ring oracle;
- optional very-strong ring labels;
- full knot or link recognition beyond explicit bounded certificates;
- arbitrary Steiner-surface search without resource bounds;
- the optional negative-curvature natural-tiling waist rule;
- natural tilings for disconnected, one-periodic, or two-periodic nets;
- noncrystallographic or barycentric-collision symmetry cases beyond the first collision-free backend;
- full abstract periodic-net isomorphism across arbitrary cells, bases, supercells, and atom orderings;
- incremental reuse of symmetry, strength, face, or tiling data after a ring-bound increase;
- incremental local ring, surface, or tiling updates after topology changes;
- arbitrary branched linker-component contraction;
- automatic atom-identity mapping between independently ordered files;
- topology-aware caching across processes;
- refactoring the mature atomic point-cell list and the new extended-object linked-cell backend into one generic spatial engine before two concrete implementations demonstrate identical invariants;
- a fully generic two-body geometric candidate generator shared by atoms, graph objects, surfaces, and tiles beyond the already shared Verlet-validity kernel;
- compiled or GPU graph and surface search;
- inferred ring lineage when canonical keys differ;
- automatic rough-interface reconstruction and learned phase classification; and

Deferral is deliberate. A deferred feature is promoted only after a scientific need, a precise definition, a finite failure policy, and a validation fixture exist.

Accepted architectural decisions

The following decisions are the current baseline:

1. `AtomisticFrameCollection` remains connectivity- and topology-agnostic.
2. Connectivity, framework topology, net projection, symmetry, embedding, tiling, and chemical interpretation are separate scientific states.
3. `FrameworkTopology` preserves decorated periodic multigraph structure, including parallel edges and linker paths.
4. The first `PeriodicNetView` backend preserves the exact framework vertex/edge orbit sets and changes only automorphism signatures.
5. Any future graph filtering, contraction, or edge removal defines a new graph and requires a compatible primitive-ring catalog.
6. Ignoring labels permits edge or vertex exchange; it never merges parallel edge orbits.
7. The first natural-tiling backend requires one connected three-periodic net.
8. `PeriodicNetEmbedding` is distinct from `PeriodicNetView` and from a trajectory frame.
9. Natural-tiling topology is constructed only on a validated symmetry-compatible reference embedding.
10. Distorted trajectory frames do not redefine the tiling.
11. Projected net-edge geometry is explicit; decorated atomic linker paths do not silently define topological face boundaries.
12. The default ring family is untruncated `PRIMITIVE_NO_SHORTCUT` from `SHORTEST_PATH_PAIRS`.
13. The removed-edge method remains the narrower `EDGE_SHORTEST_SUBSET`.
14. Local rings require lifted simplicity, exact closure, and zero winding.
15. The periodic shortest-path-pair algorithm is complete through the requested untruncated bound.
16. No independent LTA-specific 8-ring enumerator is permitted.
17. `PrimitiveRingCatalog` remains the sole persistent scientific ring result at Stage 5.
18. Physical ring placement and cyclic boundary parametrization are separate objects.
19. `RingPlacement.image_shift` is relative to the deterministic canonical lifted representative associated with the `PrimitiveRingKey`.
20. Reverse boundary indexing follows the explicit vertex/edge formulas in this manual.
21. Persistent ring identity uses `(topology graph digest, PrimitiveRingKey)`; `PeriodicNetView` and `PeriodicNetEmbedding` digests are additional provenance.

22. Dense `ring_id` is local and unstable under bound refinement.
23. A ring key is stable only after deterministic framework gauge normalization; arbitrary cells, bases, indices, and supercells require explicit mappings.
24. `LiftedAtomRef` and `LiftedVertexRef` retain distinct semantic types.
25. Stage 5 uses exact physical lifted-edge support; quotient-edge vectors are diagnostic/local encodings only.
26. A public oriented chain algebra is deferred to the actual cell-complex layer.
27. `PrimitiveRingIndex` is source-bound, nonserialized, and built from the ring catalog alone; topology-dependent consumers use a validated context.
28. Translated placements and shared-boundary relations are generated lazily.
29. Global ring-contact graphs are not part of the first architecture.
30. Pairwise contact is not a complete face-compatibility model.
31. Stage-5 helpers are extracted only after symmetry and strength prototypes.
32. The authoritative symmetry is the automorphism group of `PeriodicNetView`; every accepted operation is bound to the exact view digest, not a distorted-frame space group.
33. The infinite group is represented by a translation lattice plus finite coset representatives.
34. Every periodic multigraph automorphism carries explicit vertex and edge actions and a deterministic shift gauge.
35. Core `PeriodicNetSymmetry` contains only the net group; primitive-ring actions are a separate derived index bound to both symmetry and exact ring-catalog digests.
36. Induced ring actions preserve ordered vertex/step occurrence maps and cocycle-corrected group composition; arbitrary token alignments are forbidden.
37. Conventional LTA tiling uses the unlabeled T-net signature policy; chemically decorated symmetry is a separate result.
38. Normalized periodic automorphism representatives carry an exact common-translation composition cocycle; quotient multiplication alone is insufficient for absolute lifted placements.
39. The first automatic symmetry-discovery backend is complete only for its declared stable barycentric/star-frame domain and fails transactionally outside that domain.
40. Primitive and strong rings are distinct.
41. Smaller primitive rings suffice as components of any smaller-cycle decomposition.
42. Natural-tiling strength analysis requires a lower-closed primitive-ring catalog through the active upper bound.
43. A strength search domain is mathematical and immutable; resource limits are separate execution controls.
44. `STRONG_IN_DOMAIN` is bounded, witnessable, and never promoted to global strength.
45. Unresolved strength propagates to face and tiling certification.
46. Persistent strength results store candidate-set digests and compact witnesses, while exhaustive candidate workspaces remain transient.
47. Source-validated strength deserialization independently re-enumerates and verifies the declared finite theorem.
48. Exact finite cancellation has explicit support-matrix and provenance-bit resource guards.
49. Scientific `FacePlacement` identity is ring placement plus face orientation; auxiliary spanning-disk/triangulation choices are `FaceEmbeddingWitness` certificates, not separate faces by default.
50. Symmetry acts on scientific faces; equivalent witnesses may differ in auxiliary triangulation combinatorics.
51. Nonzero algebraic intersection of an oriented ring with a spanning surface of another ring is a rigorous linking certificate; zero linking number is not a complete unlinking theorem.
52. Intersection of two particular spanning-disk witnesses means those witnesses are incompatible, not automatically that the boundary rings are intrinsically catenated.
53. Disjoint embedded disk witnesses certify unlinking for the supported two-component disk-bounding case; bounded failure remains `UNRESOLVED`.
54. Face compatibility may require higher-order finite constraints.
55. Periodic geometric image reachability uses complete query-specific translation stencils from continuous lifted support bounds; fixed image shells are not correctness assumptions.
56. Large-cell extended-object geometry uses a dedicated linked-cell broad phase with automatically chosen subcells, image-labelled conservative multi-bin occupancy, and explicit self-image support.
57. The atomic and extended-object cell lists remain separate until concrete implementations justify a shared lower-level backend.

58. The existing deformation-aware Verlet validity theorem/kernel is reused for extended objects by tracking their point-like vertices, but the generic kernel must not assume unique-image/MIC semantics.
59. One scientific face is an oriented primitive-ring placement on one exact periodic embedding; auxiliary triangulations are verification witnesses.
60. The first finite disk family exhausts boundary-vertex triangulations only; finite failure is unresolved and never a knot theorem.
61. Framework penetration invalidates witness admissibility but does not erase the embedded spanning surface needed for linking certificates.
62. Shared-boundary contacts are ignored in linking sums only when the exact geometric contact is confined to the actual common lifted vertex or edge.
63. Pair status orders rigorous nonzero linking before witness incompatibility, unresolved degeneracy, prescribed shared boundary, and disjoint-disk unlinking.
64. Face realizability is a finite constraint system that may contain irreducible higher-order forbidden witness tuples.
65. For fixed-connectivity PL objects, maximum vertex displacement bounds whole- object displacement, and explicit-image object-object distance changes by at most the sum of the two vertex-displacement bounds.
66. The global sufficient distance-buffer cache condition is $2\delta_{\max} < r_{\text{skin}}$; a sharper per-object condition may use $\delta_A + \delta_B$.
67. Non-distance exact predicates may reuse the same cache kernel only through a proved buffered conservative support rule.
68. Variable-cell extended-object reuse uses the same singular-value/nonaffine margin structure as the implemented S3 atomic cache.
69. Object topology, mesh/witness identity, support rule, request identity, or embedding changes invalidate the extended-object candidate cache.
70. A periodic cell complex owns formal integer-coefficient chain algebra and translation-labelled attaching maps.
71. The quotient complex must support self-image incidence and satisfy $N_0 - N_1 + N_2 - N_3 = 0$ for a three-torus decomposition.
72. Tile overlap means interior-volume intersection; prescribed shared faces/edges/vertices are allowed, while improper crossing or containment overlap is invalid.
73. Local face-sector propagation is provisional until proved complete under explicit preconditions or independently partition-certified.
74. Volume closure is a diagnostic, not a no-void/no-overlap proof.
75. `PeriodicPartitionCertificate` is separate from scientific `PeriodicCellComplex` identity and proves complete coverage, disjoint interiors, and face conformity.
76. Properness is tested on scientific faces, attaching maps, and tile orbits against the exact `PeriodicNetView` automorphism group; auxiliary certificate meshes need not share identical symmetry combinatorics.
77. Natural selection prunes by symmetry and compatibility before cell construction.
78. Natural-tiling scientific outcome (`UNIQUE`, `MULTIPLE`, `NONE`) is separate from multidimensional certification.
79. Essential rings are assigned only as faces of an accepted tiling.
80. Increasing the primitive-ring bound rebuilds every dependent result; stable keys are used only for comparison and reporting.
81. The first Stage-10B generator is complete only relative to one exact master tetrahedral refinement and fixed witness assignment.
82. Removing master interfaces uses translation-labelled tetrahedron adjacency; a nonzero accumulated translation cycle certifies a noncompact lifted region.
83. Natural splitting retains all inclusion-maximal viable strong-face selections; incomparable alternatives remain explicit and enumeration order is irrelevant.
84. Ring geometry and ring-site analysis remain a parallel branch.
85. Natural tile topology and frame-dependent tile geometry remain separate.
86. Natural tiles, accessible cages, topological windows, and accessible portals are distinct objects.
87. Every rejection, truncation, ambiguity, conditional result, and unresolved state retains a machine-readable witness or diagnostic.
88. Primitive-cell/supercell comparison requires an explicit periodic-net map and is not an early Stage-5 gate.
89. The LTA domain target remains the published three tile types in ratio $3 : 1 : 1$.

90. External methods are cited where adopted; original `mdstats` derivations are identified separately.
91. Each scientific module receives a Markdown/PDF specification, implementation, audit, and test gate before the next dependent module begins.
92. Stage-5 finite cancellation uses complete physical `LiftedEdgeInstanceRef` basis elements; quotient-equivalent edges at different translations never cancel.
93. `NOT_IN_SUPPLIED_SPAN` is exact only for the supplied finite candidate set and is never promoted to `STRONG` or `STRONG_IN_DOMAIN` without a separately certified exhaustive strength domain.
94. Positive finite cancellation carries an exact re-verifiable placement witness; public periodic chain algebra remains deferred to the cell-complex layer.
95. Properness is certified only from a complete exact symmetry-discovery record; a caller-supplied subgroup cannot establish equality of automorphism groups.
96. Scientific face and tile actions exclude auxiliary triangulations and tetrahedral partition meshes; those objects certify a realization but do not define tiling identity.
97. Natural-tiling candidate identity and evidence identity use separate digests, so different exact auxiliary certificates cannot create duplicate tilings.
98. `NONE`, `UNIQUE`, and `MULTIPLE` count eligible scientific identities only; unresolved and rejected candidates remain explicit and enumeration order is never a scientific tie-breaker.
99. The natural tile graph is a species-independent structural scaffold; the species-dependent kinetic graph is a periodic directed multigraph of metastable site states.
100. Every ring owns two persistent oriented ring–tile side anchors, but physical site nodes may number zero, one, two, several, or form a continuous annulus.
101. Generic ring-site semantics use ring order plus adjacent tile types; conventional framework labels decorate, but never replace, scientific identities.
102. Accessibility is derived from a species, temperature, kinetic model, initial condition, observation time, and probability/rate criterion; it is not an intrinsic Boolean ring label.
103. Forward and reverse rates derive from shared transition-state and state free energies rather than independently fitted barriers whenever thermodynamic consistency is claimed.
104. Ring breathing is modeled in separate fast-gating, slow-gating, and comparable- timescale regimes; failure of a Markov reduction remains explicit.
105. Literature kinetic defaults are provenance-bound probability distributions or intervals with transferability conditions, not universal constants by ring order.

Theoretical and algorithmic references

82. Chung, S. J., Hahn, Th., and Klee, W. E. (1984). *Nomenclature and Generation of Three-Periodic Nets: The Vector Method.* Acta Crystallographica Section A, 40, 42-50. DOI: 10.1107/S0108767384000088.

83. Klee, W. E. (2004). *Crystallographic Nets and Their Quotient Graphs.* Crystal Research and Technology, 39(11), 959-968. DOI: 10.1002/crat.200410281.

84. Horton, J. D. (1987). *A Polynomial-Time Algorithm to Find the Shortest Cycle Basis of a Graph.* SIAM Journal on Computing, 16(2), 358-366. DOI: 10.1137/0216026.

85. Vismara, P. (1997). *Union of All the Minimum Cycle Bases of a Graph.* The Electronic Journal of Combinatorics, 4(1), R9. DOI: 10.37236/1294.

86. Goetzke, K., and Klein, H. J. (1991). *Properties and Efficient Algorithmic Determination of Different Classes of Rings in Finite and Infinite Polyhedral Networks.* Journal of Non-Crystalline Solids, 127, 215-220.

87. Yuan, X., and Cormack, A. N. (2002). *Efficient Algorithm for Primitive Ring Statistics in Topological Networks.* Computational Materials Science, 24(3), 343-360. DOI: 10.1016/S0927-0256(01)00256-7.

88. Delgado-Friedrichs, O., and O'Keeffe, M. (2003). *Identification of and Symmetry Computation for Crystal Nets.* Acta Crystallographica Section A, 59, 351-360. DOI: 10.1107/S0108767303012017.

89. Delgado-Friedrichs, O. (2003). *Barycentric Drawings of Periodic Graphs.* In G. Liotta (Ed.), Graph Drawing 2003, Lecture Notes in Computer Science 2912, 178-189. DOI: 10.1007/978-3-540-24595-7_17.

90. Blatov, V. A., Delgado-Friedrichs, O., O'Keeffe, M., and Proserpio, D. M. (2007). *Three-Periodic Nets and Tilings: Natural Tilings for Nets*. Acta Crystallographica Section A, 63, 418-425. DOI: 10.1107/S0108767307038287.

91. Anurova, N. A., Blatov, V. A., Ilyushin, G. D., and Proserpio, D. M. (2010). *Natural Tilings for Zeolite-Type Frameworks*. Journal of Physical Chemistry C, 114, 10160-10170. DOI: 10.1021/jp1030027.

92. Delgado-Friedrichs, O., O'Keeffe, M., Proserpio, D. M., and Treacy, M. M. J. (2023). *Three-Periodic Nets, Tilings and Surfaces: A Short Review and New Results*. Acta Crystallographica Section A, 79, 192-202. DOI: 10.1107/S2053273323000414.

93. Shewchuk, J. R. (1997). *Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates*. Discrete & Computational Geometry, 18, 305-363. DOI: 10.1007/PL00009321.

94. Moller, T. (1997). *A Fast Triangle-Triangle Intersection Test*. Journal of Graphics Tools, 2(2), 25-30. DOI: 10.1080/10867651.1997.10487472.

95. Hass, J., Snoeyink, J., and Thurston, W. P. (2003). *The Size of Spanning Disks for Polygonal Curves*. Discrete & Computational Geometry, 29, 1-17. DOI: 10.1007/s00454-002-2824-1.

96. Verlet, L. (1967). *Computer "Experiments" on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules*. Physical Review, 159, 98-103. DOI: 10.1103/PhysRev.159.98.

97. Chialvo, A. A., and Debenedetti, P. G. (1990). *On the Use of the Verlet Neighbor List in Molecular Dynamics*. Computer Physics Communications, 60, 215-224.

98. Quentrec, B., and Brot, C. (1973). *New Method for Searching for Neighbors in Molecular Dynamics Computations*. Journal of Computational Physics, 13(3), 430-432. DOI: 10.1016/0021-9991(73)90046-6.

99. Hsieh, C.-C., Kauffman, L. H., and Tsau, C.-M. (2017). *A Combinatorial Algorithm for Computing Higher Order Linking Numbers*. Asian Journal of Mathematics, 21(2), 265-286. DOI: 10.4310/AJM.2017.v21.n2.a3.

100. Newman, M. (1972). *Integral Matrices*. Academic Press, New York. The determinant-divisor/Smith-normal-form treatment supplies the full-rank integer-sublattice index used by `PeriodicNetView`.

101. Eyring, H. (1935). *The Activated Complex in Chemical Reactions*. Journal of Chemical Physics, 3, 107-115. DOI: 10.1063/1.1749604.

102. Vineyard, G. H. (1957). *Frequency Factors and Isotope Effects in Solid State Rate Processes*. Journal of Physics and Chemistry of Solids, 3, 121-127. DOI: 10.1016/0022-3697(57)90059-8.

103. Kramers, H. A. (1940). *Brownian Motion in a Field of Force and the Diffusion Model of Chemical Reactions*. Physica, 7, 284-304. DOI: 10.1016/S0031-8914(40)90098-2.

104. Haenggi, P., Talkner, P., and Borkovec, M. (1990). *Reaction-Rate Theory: Fifty Years after Kramers*. Reviews of Modern Physics, 62, 251-341. DOI: 10.1103/RevModPhys.62.251.

105. Zwanzig, R. (1992). *Dynamical Disorder: Passage through a Fluctuating Bottleneck*. Journal of Chemical Physics, 97, 3587-3589. DOI: 10.1063/1.462993.

106. Gillespie, D. T. (1977). *Exact Stochastic Simulation of Coupled Chemical Reactions*. Journal of Physical Chemistry, 81, 2340-2361. DOI: 10.1021/j100540a008.

107. Fichthorn, K. A., and Weinberg, W. H. (1991). *Theoretical Foundations of Dynamical Monte Carlo Simulations*. Journal of Chemical Physics, 95, 1090-1096. DOI: 10.1063/1.461138.

108. Metzner, P., Schuette, C., and Vanden-Eijnden, E. (2009). *Transition Path Theory for Markov Jump Processes*. Multiscale Modeling & Simulation, 7, 1192-1219. DOI: 10.1137/070699500.

109. Faradjian, A. K., and Elber, R. (2004). *Computing Time Scales from Reaction Coordinates by Milestoning*. Journal of Chemical Physics, 120, 10880-10889. DOI: 10.1063/1.1738640.

110. Prinz, J.-H., Wu, H., Sarich, M., Keller, B., Senne, M., Held, M., Chodera, J. D., Schuette, C., and Noe, F. (2011). *Markov Models of Molecular Kinetics: Generation and Validation*. Journal of Chemical Physics, 134, 174105. DOI: 10.1063/1.3565032.

111. Henkelman, G., Uberuaga, B. P., and Jonsson, H. (2000). *A Climbing Image Nudged Elastic Band Method for Finding Saddle Points and Minimum Energy Paths*. Journal of Chemical Physics, 113, 9901-9904. DOI: 10.1063/1.1329672.

112. Torrie, G. M., and Valleau, J. P. (1977). *Nonphysical Sampling Distributions in Monte Carlo Free-Energy Estimation: Umbrella Sampling*. Journal of Computational Physics, 23, 187-199. DOI: 10.1016/0021-9991(77)90121-8.

References [1, 2] establish periodic quotient-graph representation. References [3-6] provide shortest-path cycle and primitive-ring foundations. References [7, 8] provide exact periodic-net symmetry and topology-derived barycentric placement. References [9-11] define natural, zeolite, and essential-ring tiling concepts. References [12-14] support robust embedded-face predicates and bounded spanning-disk semantics. References [15,16] provide the classical Verlet buffering and displacement-triggered neighbor-list foundation reused by the extended-object cache-validity design. Reference [17] supplies the classical linked-cell neighbor search adapted by the extended-object broad phase. Reference [18] provides an intersection-theoretic algorithmic treatment of linking and higher-order linking; `mdstats` uses the standard algebraic ring–spanning-surface intersection as a certificate without treating zero linking number as a complete unlinking test. Reference [19] supplies the determinant-divisor/Smith-normal-form fact used to convert full-rank cycle-gain generators into a finite translation-subgroup index. References [101-112] supply the rate-theory and stochastic-kinetics foundations for the revised Stage-11 plan: transition-state theory, harmonic solid-state prefactors, dissipative barrier crossing, fluctuating bottlenecks, exact jump simulation, kinetic Monte Carlo, transition-path analysis, milestoning, Markov model validation, NEB saddle searches, and umbrella free-energy sampling.

The bounded periodic completeness theorem, finite-radius reduction H_K, explicit signature-only first net-view backend, canonical ring-placement anchor, source-bound ring indexing, symmetry-compatible embedding split, continuous-lift periodic translation-stencil/extended-cell-list architecture, query-specific support rules, multi-image Verlet-kernel adaptation, the fixed-connectivity PL vertex-displacement proof, scientific-face versus embedding-witness separation, certification-state propagation, and integration with trajectory topology classes are `mdstats` contributions. The local face-side propagation and periodic partition strategies remain provisional architectural proposals; any external symmetry, surface, or partition algorithm adopted in implementation will be cited in the corresponding specification and code.

Context-restoration checklist

Before implementing or revising this stack, recover the following context:

- Which `FrameworkTopology`, deterministic periodic gauge, and `PeriodicNetView` own the result?
- Which vertex and edge signatures does the active `NetViewPolicy` preserve?
- Does the net have one quotient component, translation rank three, and translation-subgroup index one?
- Which primitive method, family, bound, and resource diagnostics produced the ring catalog?
- Is the cycle identified by stable ring key or only by one catalog-local ID?
- Is the object a `RingPlacement`, `CycleParameterization`, or oriented boundary view?
- Is exact boundary support expressed in physical lifted-edge instances?
- Has a ring-bound increase invalidated downstream symmetry, strength, face, or tiling data?
- Does an automorphism include explicit edge action and deterministic shift gauge?
- Is any primitive-ring action bound separately to the exact symmetry and primitive-ring catalog digests?
- Which reusable `PeriodicBarycentricPlacement` owns the exact rational coordinates, and are collisions/resource bounds recorded?
- What immutable `RingStrengthDomain` candidate placement domain was exhausted, and were any separate execution resources truncated?
- Is the exhaustive strength candidate set transient, with only its digest and verified certificate persisted?
- Which exact `PeriodicNetEmbedding` owns face topology and projected-edge geometry?
- Does each selected face include a surface-option identity and orientation?
- Are compatibility conditions pairwise, higher-order, symmetry-linked, or unresolved?
- Does the periodic cell complex retain translation-labelled attaching maps and satisfy both boundary identities?
- Is no-overlap/no-void supported by an explicit partition certificate rather than volume alone?
- Has properness been certified against the exact net-view automorphism group?
- What is the scientific tiling outcome (`UNIQUE`, `MULTIPLE`, or `NONE`), and which independent certification dimensions remain bounded, unresolved, or truncated?

- Are physical descriptors evaluated only on frames compatible with the persistent topology and tiling identities?
- Are the two ring-side anchors being kept distinct from the number of physical microstates?
- Which species, local chemistry, occupancy state, temperature, and geometric descriptors define the site landscape and rate law?
- Is breathing fast, slow, or comparable to hopping, and is a Markov reduction actually justified?
- Which external method is adapted, and which part is an original `mdstats` derivation?

Final architecture summary

The implemented and planned architecture is

```
atomic coordinates
      -> atomic connectivity
      -> decorated periodic framework topology
             |             | +-> PeriodicNetView
             |             |       -> exact barycentric placement
             |             |       -> core exact net symmetry
             |             |              -> catalog-bound primitive-ring symmetry index
             |             |       -> PeriodicNetEmbedding
             |
             `-> complete bounded primitive rings
                    -> stable ring placements / parametrizations
                    -> source-bound ring index
                    -> transient strength-search workspace
                    -> persistent verified bounded strength result

PeriodicNetEmbedding + eligible strong rings
      -> periodic spatial broad phase (translation stencil / extended cell list)
      -> scientific face placements + embedding witnesses/constraints
      -> periodic cell complex
      -> separate PeriodicPartitionCertificate
      -> natural-tiling outcome + multidimensional certification
```

The downstream physical and kinetic branch is

```
primitive rings + natural tiling
      -> ring geometry + tile geometry
      -> framework semantic registry
      -> species-dependent site microstates
      -> periodic parameterized transition network
      -> breathing-conditioned kinetic realization
      -> residence, first-passage, pathway, and diffusion statistics
```

The natural tile graph remains the structural scaffold. The site-state network is the kinetic graph.

Stage 5 adds no second scientific ring catalog and no premature public chain algebra. The geometric branch adds a separate symmetry-compatible net embedding, a dedicated extended-object periodic broad phase, and reuse of the proven deformation-aware Verlet validity kernel through vertex displacement bounds. The atomic and extended-object cell lists remain separate; only the mathematically shared cache-validity kernel is factored initially. Stage 5 establishes exact source-bound placement, parametrization, identity, and inverse-incidence operations only after concrete symmetry and strength prototypes demonstrate the need.

The architecture refuses seven invalid promotions:

101. a decorated framework topology is not automatically the net whose symmetry defines properness;
102. a catalog-local ring ID is not a stable cross-bound identity;
103. a quotient-edge vector is not an exact physical ring boundary;
104. a primitive ring is not automatically strong;
105. a boundedly strong ring is not automatically an embedded face;
106. an auxiliary disk triangulation is not a distinct scientific face merely because its witness mesh differs; and
107. a natural tile is not automatically an accessible cage;

108. two topological ring sides do not automatically imply two physical site minima;
109. a geometric aperture is not automatically a kinetic barrier or a rate;
110. a static barrier is not automatically valid under ring breathing; and
111. a finite-state Markov model is not accepted without a timescale and memory justification.

The primitive-ring foundation is not provisional. Under the documented untruncated assumptions, the periodic shortest-path-pair algorithm is complete through its requested bound. The exact first-backend net view, multigraph-aware symmetry, bounded strength classification, embedded-face certificates, periodic cell-complex and partition certificates, properness checks, finite master-refinement natural-face search, and explicit primitive-bound full-rebuild comparison are now implemented. Remaining Part I work includes automatic master-refinement construction for general nets, automatic tetrahedralization, local face-sector discovery, and general partial-periodic support. The exact LTA reference tile geometry, topological windows, conservative cage/portal witnesses, accessibility rank, compatible-frame geometry, persistent and dynamic T/O ring geometry, atom-resolved serrated boundaries, and framework semantics are implemented. Species-dependent statistical sites and every kinetic branch are Part II responsibilities and remain subject to its explicit scientific gates.

Optional MLFF material-profile boundary

This framework/ring architecture is not a default MLFF feature benchmark. It is activated only for systems with a meaningful periodic framework, porous network, zeolite, cage, channel, or ring topology. Most crystalline solids, amorphous solids, liquids, and interfaces require only the general structural observables owned by `structural_observables_architecture.{md,pdf}`.

An MLFF profile may call Part I APIs to obtain stable framework, ring, window, cage, or geometric identities and may call Part II for site/path evidence. The MLFF branch must not recreate ring enumeration, ring centers, tilings, cage geometry, or site semantics. Profile activation and call provenance belong to MLFF; scientific topology and geometry remain owned here.

MLFF optional-extension integration

Framework, ring, cage, window, and site algorithms remain owned by this architecture and its specialized provider modules. Beginning with MLFF-DATA9A7d in `mdstats 0.20.50a0`, the MLFF training-data branch does not embed those scientific schemas as generic DATA4 or DATA6 fields. Instead it stores the provider result in a `ProfileFeatureCatalog` envelope carrying extension, stage, provider, frame, and parent-bundle lineage.

For LTA, the extension ID is `lta` and activation requires the declared `porous_network -> zeolite -> lta` hierarchy. The adapter may expose a namespaced frame vector, atomic environments, and environment-class labels for MLFF selection. The definitions of rings, sites, crossings, cages, and windows remain authoritative here; the MLFF branch must not reinterpret them.